

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TIẾN ĐỘ

Đề tài:

**Xây dựng hệ thống phân tầng mức độ rủi ro học
tập của sinh viên**

Giảng viên hướng dẫn : TS. Nguyễn Thị Mai Trang

Sinh viên thực hiện : Đàm Chiến Thắng - B23DCKH106

: Đỗ Quốc Tấn - B23DCKH104

Học phần : Thực tập cơ sở - INT13147

Lớp học phần : Nhóm 17

Hà Nội – 2026

LỜI MỞ ĐẦU

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trong lĩnh vực giáo dục, dữ liệu học tập của sinh viên ngày càng được thu thập và lưu trữ với khối lượng lớn thông qua các nền tảng học trực tuyến (LMS). Tuy nhiên, việc khai thác hiệu quả nguồn dữ liệu này nhằm hỗ trợ quản lý và nâng cao chất lượng đào tạo vẫn còn nhiều hạn chế, đặc biệt trong công tác phát hiện sớm sinh viên có nguy cơ học tập yếu kém hoặc bỏ học.

Với quy mô lớp học ngày càng đông, giảng viên gặp nhiều khó khăn trong việc theo dõi sát sao tình trạng học tập của từng sinh viên, dẫn đến các quyết định hỗ trợ thường mang tính thụ động và dựa nhiều vào kinh nghiệm cá nhân thay vì dữ liệu thực tế. Thực trạng đó đặt ra nhu cầu cấp thiết về một hệ thống tự động, có khả năng phân tích hành vi học tập và đưa ra cảnh báo sớm dựa trên bằng chứng dữ liệu.

Xuất phát từ bối cảnh đó, nhóm nghiên cứu đề xuất đề tài **"Xây dựng hệ thống phân tầng mức độ rủi ro học tập của sinh viên"** ứng dụng các kỹ thuật Học máy (Machine Learning) để phân loại sinh viên vào 4 mức độ rủi ro: Thấp, Trung bình, Cao và Rất cao, dựa trên bộ dữ liệu giáo dục mở OULAD (Open University Learning Analytics Dataset).

Hệ thống được xây dựng với mong muốn đóng vai trò như một công cụ hỗ trợ ra quyết định, giúp nhà trường và giảng viên chủ động phát hiện và can thiệp kịp thời với những sinh viên có nguy cơ, từ đó giảm thiểu tỷ lệ trượt môn và cải thiện chất lượng đào tạo.

Hà Nội, tháng 5 năm 2026

Nhóm tác giả

Đàm Chiến Thắng — Đỗ Quốc Tấn

MỤC LỤC

LỜI MỞ ĐẦU.....	2
MỤC LỤC.....	3
DANH MỤC CHỮ VIẾT TẮT.....	5
GIỚI THIỆU.....	6
1. Lý do chọn đề tài.....	6
2. Mục tiêu nghiên cứu.....	6
2.1. Mục tiêu tổng quát.....	6
2.2. Mục tiêu cụ thể.....	7
3. Đối tượng và phạm vi nghiên cứu.....	7
3.1. Đối tượng nghiên cứu.....	7
3.2. Phạm vi nghiên cứu.....	8
4. Phương pháp nghiên cứu.....	8
5. Bố cục báo cáo.....	8
6. Bảng phân chia công việc.....	9
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	10
1.1. Tổng quan về Học máy (Machine Learning).....	10
1.1.1. Khái niệm cơ bản.....	10
1.1.2. Quy trình xây dựng hệ thống Học máy.....	10
1.2. Học máy có giám sát (Supervised Learning).....	10
1.3. Vấn đề dữ liệu mất cân bằng (Imbalanced Dataset).....	10
1.4. Các mô hình máy học được sử dụng.....	11
1.5. Thang đo độ chính xác, hiệu suất và hiệu quả của mô hình máy học.....	11
CHƯƠNG 2: PHƯƠNG PHÁP NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH MÁY HỌC.....	12
2.1. Xác định bài toán và dữ liệu.....	12
2.1.1. Phát biểu bài toán bài toán.....	12
2.1.2. Nguồn dữ liệu (Data Source).....	12
2.2. Xây dựng pipeline xử lý dữ liệu.....	13
2.2.1. Làm sạch dữ liệu.....	13
2.2.1.1. Làm sạch bảng StudentInfo.....	14
2.2.1.2. Xử lý missing score và is_banked bảng StudentAssessment.....	15
2.2.1.3. Xử lý bảng studentRegistration.....	15
2.2.2. Mã hóa biến.....	16
2.2.3. Feature Engineering.....	16
2.2.4. Kết nối bảng.....	20
2.2.5. Gán nhãn mức rủi ro (4 lớp).....	22

2.2.6. Train/ Validation/ Test split (stratified 60/20/20).....	23
2.2.7. Chuẩn hóa dữ liệu (StandardScaler).....	24
2.3. Xử lý dữ liệu mất cân bằng.....	24
2.4. Mô hình máy học phù hợp.....	25
2.5. Phương pháp đánh giá.....	27
CHƯƠNG 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	28
3.1. Môi trường thực nghiệm.....	28
3.2. Kết quả phân tích dữ liệu.....	28
3.2.1. Phân tích các yếu tố chi phối rủi ro sinh viên (EDA).....	29
3.2.1.1. Đặc trưng dân số học (Demographics).....	29
3.2.1.2. Ảnh hưởng của nền tảng VLE (Mật độ Clicks).....	30
3.2.1.3. Kết quả Assessment theo mức độ rủi ro.....	32
3.2.1.4. Ma trận tương quan đặc trưng (Feature Correlation):.....	33
3.2.1.5. Phân phối đặc trưng và ngưỡng outlier (IQR).....	35
3.2.2. Vấn đề mất cân bằng dữ liệu (Imbalance Data).....	35
3.3. Kết quả huấn luyện mô hình.....	36
3.3.1. So sánh F1-score.....	37
3.3.2. So sánh AUC - ROC.....	39
3.3.3. So sánh chiến lược Imbalance.....	40
3.4. Đánh giá mô hình trên tập Test.....	40
3.4.1. Kết quả tổng thể.....	40
3.4.2. Đánh giá theo từng lớp.....	41
3.5. Đánh giá trực quan.....	41
3.6. Kết luận.....	43
CHƯƠNG 4: XÂY DỰNG HỆ THỐNG.....	45
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	45

DANH MỤC CHỮ VIẾT TẮT

GIỚI THIỆU

1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trong lĩnh vực giáo dục, dữ liệu học tập của sinh viên ngày càng được thu thập và lưu trữ với khối lượng lớn. Tuy nhiên, trên thực tế, việc khai thác và sử dụng hiệu quả các dữ liệu này nhằm hỗ trợ quản lý và nâng cao chất lượng đào tạo vẫn còn nhiều hạn chế. Bên cạnh đó, với số lượng sinh viên lớn, giảng viên gặp nhiều khó khăn trong việc theo dõi sát sao tình trạng học tập của từng cá nhân, đặc biệt là trong các học phần có quy mô lớp học đông. Các quyết định hỗ trợ học tập vì vậy thường mang tính thụ động và dựa nhiều vào kinh nghiệm cá nhân, chưa tận dụng đầy đủ dữ liệu sẵn có.

Trước thực trạng đó, làm thế nào nhà trường cảnh báo sớm với sinh viên kết quả học tập để sinh viên nhanh chóng điều chỉnh việc học tập của mình tránh tình trạng “sự việc đã rồi”. Do đó, cần có một nghiên cứu mới đầy đủ để tạo ra ứng dụng với độ chính xác cao, tin cậy để phục vụ việc dự báo sinh viên thuộc diện cảnh báo trong những học kì của năm học tới.

2. Mục tiêu nghiên cứu

2.1. Mục tiêu tổng quát

Nghiên cứu ứng dụng các kỹ thuật Học máy để xây dựng mô hình dự báo, từ đó phát triển một *Hệ thống cảnh báo sớm và phân tầng 4 mức độ rủi ro học tập* (Thấp, Trung bình, Cao, Rất cao) cho sinh viên. Hệ thống này đóng vai trò như một công cụ hỗ trợ ra quyết định, giúp giảm thiểu tỷ lệ sinh viên trượt môn, cảnh báo học vụ và nâng cao chất lượng đào tạo.

Dự báo sinh viên diện cảnh báo học tập giúp nhà trường tìm biện pháp hạn chế tối đa sinh viên thuộc diện cảnh báo và sinh viên thôi học.

Sinh viên có thể tự tính toán nguy cơ trượt môn để kịp thời thay đổi phương pháp học tập nhằm giảm thiểu rủi ro trượt môn.

Để các ban ngành, tổ chức trong nhà trường biết được những sinh viên cần trợ giúp cũng như tìm các biện pháp giáo dục công dân sinh viên.

Xây dựng hệ thống cảnh báo nguy cơ trượt môn không chỉ báo động sớm cho sinh viên mà còn hỗ trợ phụ huynh nắm thông tin học tập của con em mình, giúp gia đình sinh viên kịp thời nhắc nhở các em và sinh viên nhanh chóng điều chỉnh học tập tránh tình trạng quá muộn.

2.2. Mục tiêu cụ thể

Tổng hợp các nghiên cứu về Khai phá dữ liệu giáo dục, các yếu tố ảnh hưởng đến kết quả học tập và các thuật toán học máy áp dụng trong dự báo rủi ro học tập.

Thu thập, làm sạch và xây dựng bộ dữ liệu huấn luyện đặc thù dựa trên dữ liệu điểm số và tiến trình học tập của sinh viên. Phân tích và xử lý dữ liệu mất cân bằng: So sánh các chiến lược để lựa chọn phương pháp tối ưu cho bài toán phân lớp.

Huấn luyện và lựa chọn mô hình: Thử nghiệm 4 thuật toán (Logistic Regression, Random Forest, XGBoost, LightGBM) và lựa chọn mô hình tốt nhất dựa trên Macro F1-score và AUC-ROC.

Hoàn thiện các thang đo đánh giá độ chính xác và hiệu năng của mô hình dự báo đa lớp, tập trung vào các metric như F1-score (Macro) và AUC, đảm bảo mô hình nhận diện tốt nhóm rủi ro cao và rất cao.

Từ kết quả dự báo của mô hình, phân tích tầm quan trọng của các đặc trưng (Feature Importance) để chỉ ra các nguyên nhân cốt lõi gây rủi ro; từ đó đề xuất các biện pháp hỗ trợ cụ thể cho sinh viên và điều chỉnh phương pháp giảng dạy cho giảng viên.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Đối tượng nghiên cứu: Bộ dữ liệu giáo dục mở OULAD (Open University Learning Analytics Dataset) gồm 32.593 bản ghi sinh viên, các yếu tố tác động đến kết quả học tập của sinh viên bao gồm: đặc điểm nhân khẩu học, hành vi tương tác trên môi trường học tập ảo (VLE) và lịch sử điểm số bài kiểm tra.
- Đối tượng sử dụng hệ thống:
 - + Sinh viên: Tra cứu và nhận cảnh báo sớm về mức độ rủi ro cá nhân, từ đó chủ động điều chỉnh phương pháp và cường độ học tập.
 - + Giảng viên/Cố vấn học thuật: Theo dõi danh sách sinh viên theo từng lớp/học phần, phát hiện các cá nhân có rủi ro "Cao" và "Rất cao" để can thiệp kịp thời.

- + Cán bộ quản lý đào tạo: Sử dụng bảng điều khiển (Dashboard) phân tích để đánh giá chất lượng học tập tổng thể và đưa ra các chính sách hỗ trợ vĩ mô.

3.2. Phạm vi nghiên cứu

- Phạm vi dữ liệu: Bộ dữ liệu OULAD, nguồn mở, được cung cấp bởi Đại học Mở Vương quốc Anh (Open University, UK) theo giấy phép CC-BY 4.0. Không thu thập dữ liệu trực tiếp từ hệ thống nội bộ của bất kỳ trường nào nhằm tuân thủ quy định bảo mật thông tin cá nhân.
- Phạm vi thời gian dữ liệu: Dữ liệu tương tác VLE được giới hạn trong 60 ngày đầu của mỗi khóa học nhằm phục vụ bài toán cảnh báo sớm và tránh rò rỉ dữ liệu (data leakage).
- Phạm vi bài toán: Tập trung vào bài toán phân loại đa lớp 4 mức rủi ro, xử lý mất cân bằng dữ liệu và tối ưu hóa hiệu năng mô hình. Đề tài không bao gồm triển khai hệ thống thương mại hóa.
- Phạm vi thời gian thực hiện: Đề tài được nghiên cứu, triển khai và đánh giá trong thời gian 5 tháng (02/2026 — 06/2026).

4. Phương pháp nghiên cứu

5. Bố cục báo cáo

Báo cáo được tổ chức theo cấu trúc 6 chương chính:

Chương	Tiêu đề	Nội dung chính
Chương 1	Cơ sở lý thuyết	Học máy, Supervised Learning, xử lý mất cân bằng, các mô hình và thang đo đánh giá
Chương 2	Phương pháp nghiên cứu và xây dựng mô hình	Xác định bài toán, pipeline xử lý dữ liệu, Feature Engineering, gán nhãn rủi ro, mô hình và phương pháp đánh giá
Chương 3	Thực nghiệm và đánh giá	Môi trường thực nghiệm, kết quả EDA, so sánh F1-score và AUC, so sánh chiến lược imbalance, đánh giá chi tiết mô hình tốt nhất, ưu nhược điểm
Chương 4	Thiết kế hệ thống	Thiết kế kiến trúc hệ thống, xây dựng giao diện web

Chương 5	Kết luận và hướng phát triển	Kết luận chung, điểm mạnh & điểm yếu, hướng phát triển
----------	------------------------------	--

6. Bảng phân chia công việc

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về Học máy (Machine Learning)

1.1.1. Khái niệm cơ bản

Học máy (Machine Learning) là một nhánh của Trí tuệ nhân tạo (AI), cho phép hệ thống tự động học từ dữ liệu và cải thiện hiệu suất theo thời gian mà không cần lập trình tường minh. Mô hình học máy tìm kiếm các quy luật ẩn trong dữ liệu đầu vào để đưa ra dự đoán hoặc quyết định trên dữ liệu mới.

Đề tài sử dụng ngôn ngữ lập trình Python với các thư viện chính: Pandas, NumPy (xử lý dữ liệu), Matplotlib, Seaborn (trực quan hóa), Scikit-learn, XGBoost, LightGBM, imbalanced-learn (xây dựng mô hình). Môi trường phát triển: Jupyter Notebook trong Anaconda.

1.1.2. Quy trình xây dựng hệ thống Học máy

- Đề tài áp dụng quy trình chuẩn 7 bước:
 1. Thu thập dữ liệu từ bộ dữ liệu OULAD.
 2. Tiền xử lý dữ liệu: làm sạch, xử lý giá trị thiếu, chuẩn hóa và mã hóa biến.
 3. Phân tích khám phá dữ liệu (EDA) và phân tích imbalance.
 4. Lựa chọn và xây dựng mô hình phân loại phù hợp.
 5. Huấn luyện mô hình trên tập train với chiến lược xử lý mất cân bằng.
 6. Đánh giá hiệu suất trên tập test bằng F1-score (Macro) và AUC.
 7. Tối ưu hóa siêu tham số.

1.2. Học máy có giám sát (Supervised Learning)

Học máy có giám sát là phương pháp huấn luyện mô hình trên tập dữ liệu có nhãn, bao gồm hai thành phần:

- Đặc trưng (Features — X): Vector đầu vào gồm các biến độc lập mô tả đặc điểm của từng sinh viên (hành vi VLE, điểm số, nhân khẩu học).
- Nhãn (Label — Y): Biến mục tiêu cần dự đoán, ở đây là mức độ rủi ro học tập $Y \in \{0, 1, 2, 3\}$.

Đề tài tập trung vào bài toán Phân lớp đa lớp (Multi-class Classification), trong đó mỗi sinh viên được gán một trong bốn mức rủi ro: Thấp (0), Trung bình (1), Cao (2) và Rất cao (3).

1.3. Vấn đề dữ liệu mất cân bằng (Imbalanced Dataset)

Dữ liệu mất cân bằng xảy ra khi số lượng mẫu thuộc một lớp áp đảo hoàn toàn so với các lớp còn lại. Trong bộ dữ liệu OULAD, lớp Very High

(Fail + Withdrawn) chiếm ~52.8%, trong khi lớp High chỉ chiếm ~1.9%. Nếu không xử lý, mô hình sẽ bị thiên kiến (bias) về phía lớp đa số, dẫn đến accuracy cao nhưng F1-score và AUC thấp trên các lớp thiểu số.

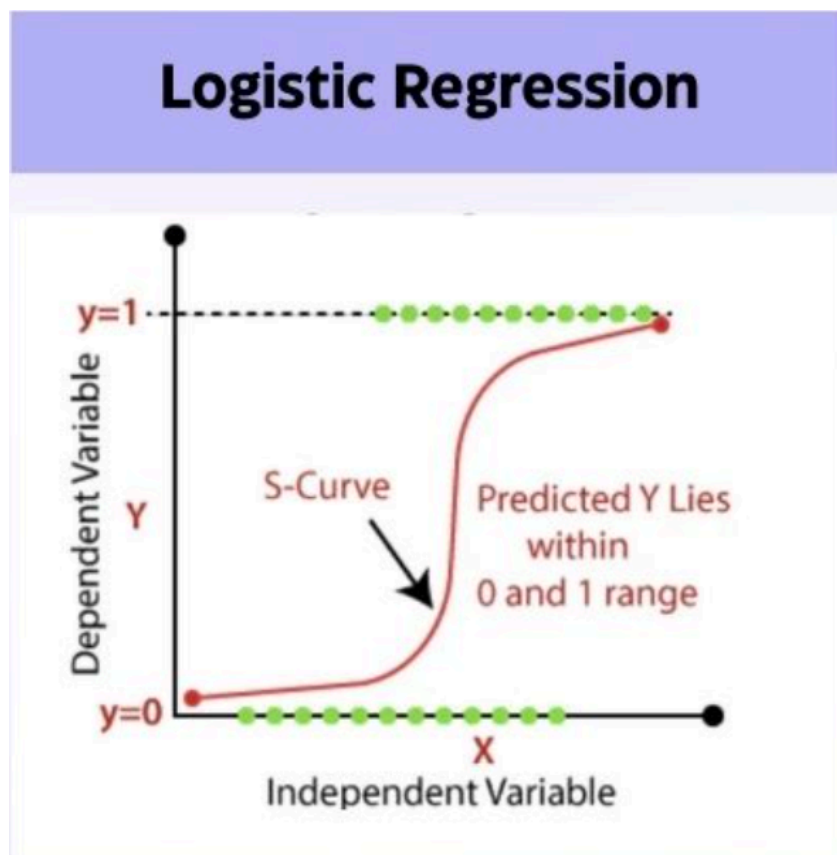
Đề tài so sánh ba chiến lược xử lý:

- Tạo mẫu tổng hợp cho lớp thiểu số bằng nội suy k-NN.SMOTE (Synthetic Minority Over-sampling Technique):
- Tăng trọng số penalty cho lớp thiểu số trong hàm loss.class_weight = 'balanced'
- Ensemble tích hợp cơ chế cân bằng lớp vào quá trình bootstrap.

1.4. Các mô hình máy học được sử dụng

Để giải quyết bài toán phân loại mức độ rủi ro của sinh viên, nhóm nghiên cứu sử dụng bốn thuật toán học máy phổ biến gồm Logistic Regression, Random Forest, XGBoost và LightGBM. Các mô hình này đại diện cho những hướng tiếp cận khác nhau trong bài toán phân loại, từ mô hình tuyến tính đơn giản đến các phương pháp Ensemble Learning hiện đại. Việc sử dụng nhiều mô hình cho phép đánh giá khách quan hiệu quả dự đoán, đồng thời lựa chọn được thuật toán phù hợp nhất với đặc điểm dữ liệu giáo dục.

1.4.1. Logistic Regression



Hình 1.1 Máy học Logistic Regression

- Khái niệm:

Logistic Regression là thuật toán học máy có giám sát được sử dụng để dự đoán xác suất một đối tượng thuộc về một lớp nhất định. Trong nghiên cứu này, mô hình được áp dụng để dự đoán sinh viên thuộc nhóm rủi ro thấp, trung bình hoặc cao dựa trên các thông tin học tập.

- Nguyên lý hoạt động:

Mô hình xây dựng một hàm tuyến tính từ các biến đầu vào như điểm GPA, tỷ lệ chuyên cần, số tín chỉ nợ hoặc số lượt tương tác với hệ thống học trực tuyến. Giá trị tuyến tính sau đó được chuyển đổi thành xác suất thông qua hàm Sigmoid.

Mô hình sẽ học trọng số của từng đặc trưng trong quá trình huấn luyện. Những thuộc tính có ảnh hưởng lớn đến kết quả dự đoán sẽ được gán trọng số cao hơn.

- + Ví dụ, một sinh viên có điểm đánh giá thấp, số lượt truy cập hệ thống học tập ít và tỷ lệ chuyên cần thấp sẽ có xác suất thuộc nhóm rủi ro cao lớn hơn sinh viên có kết quả học tập tốt.

- Ưu điểm:

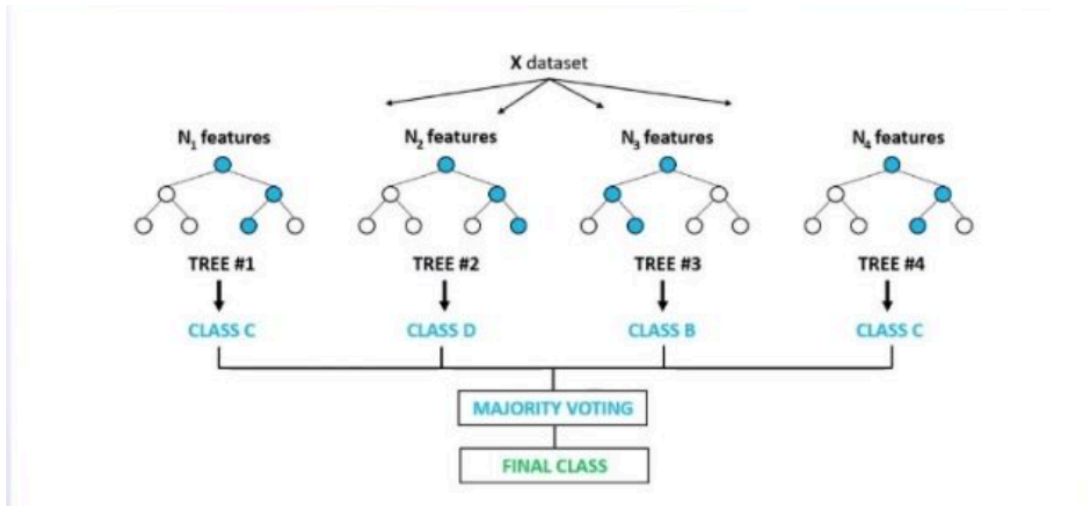
- + **Dễ triển khai và huấn luyện:** Logistic Regression có cấu trúc đơn giản và yêu cầu ít tài nguyên tính toán. Điều này giúp mô hình được huấn luyện nhanh ngay cả trên tập dữ liệu có số lượng bản ghi lớn.
- + **Dễ giải thích kết quả:** Các hệ số của mô hình cho phép đánh giá mức độ ảnh hưởng của từng đặc trưng đến kết quả phân loại. Đây là ưu điểm quan trọng trong các nghiên cứu giáo dục khi cần giải thích nguyên nhân dẫn đến rủi ro học tập.
- + **Tốc độ dự đoán nhanh:** Sau khi được huấn luyện, mô hình có thể đưa ra dự đoán gần như tức thời, phù hợp với các hệ thống cảnh báo sớm.

- Nhược điểm:

- + **Khó mô hình hóa các mối quan hệ phi tuyến:** Logistic Regression hoạt động hiệu quả khi mối quan hệ giữa các biến tương đối tuyến tính. Tuy nhiên dữ liệu giáo dục thường chứa nhiều tương tác phức tạp nên khả năng biểu diễn của mô hình bị hạn chế.
- + **Hiệu năng thấp hơn các mô hình Ensemble:** Khi dữ liệu có nhiều đặc trưng và quan hệ phức tạp, Logistic Regression thường cho độ chính xác thấp hơn Random Forest, XGBoost hoặc LightGBM.

- Vai trò trong đề tài: Logistic Regression được sử dụng làm mô hình Baseline nhằm tạo cơ sở so sánh với các thuật toán nâng cao. Kết quả của mô hình giúp đánh giá mức độ cải thiện hiệu năng của các phương pháp Ensemble Learning.

1.4.2. Random Forest



Hình 1.2. Máy học Random Forest

- Khái niệm: Random Forest là thuật toán học máy thuộc nhóm Ensemble Learning, được xây dựng bằng cách kết hợp nhiều cây quyết định khác nhau. Thay vì dựa vào một cây duy nhất, Random Forest tổng hợp kết quả từ nhiều cây để đưa ra dự đoán cuối cùng.
- Nguyên lý hoạt động: Random Forest sử dụng kỹ thuật Bagging (Bootstrap Aggregating). Từ tập dữ liệu ban đầu, thuật toán tạo ra nhiều tập dữ liệu con bằng cách lấy mẫu ngẫu nhiên có hoàn lại. Mỗi tập dữ liệu được sử dụng để huấn luyện một cây quyết định riêng biệt. Khi dự đoán, tất cả các cây sẽ bỏ phiếu và lớp nhận được nhiều phiếu nhất sẽ trở thành kết quả cuối cùng.
- Ưu điểm:
 - + **Giảm hiện tượng Overfitting:** Việc kết hợp nhiều cây quyết định giúp mô hình tránh ghi nhớ quá mức dữ liệu huấn luyện.
 - + **Độ chính xác cao:** Sai số của từng cây riêng lẻ được giảm thiểu thông qua cơ chế bỏ phiếu đa số.
 - + **Đánh giá được tầm quan trọng của đặc trưng:** Random Forest có khả năng xác định những yếu tố ảnh hưởng mạnh nhất đến nguy cơ học tập của sinh viên.
- Nhược điểm:
 - + **Tốn tài nguyên tính toán:** Do phải xây dựng và lưu trữ nhiều cây quyết định nên yêu cầu bộ nhớ và thời gian xử lý lớn hơn.

- + **Khó giải thích hơn Logistic Regression:** Việc tổng hợp kết quả từ hàng trăm cây quyết định khiến quá trình giải thích dự đoán trở nên phức tạp hơn.
- Vai trò trong đề tài: Random Forest đại diện cho nhóm thuật toán Ensemble Learning sử dụng chiến lược Bagging, được sử dụng để đánh giá khả năng khai thác các quan hệ phi tuyến trong dữ liệu sinh viên.

1.4.3. XGBoost



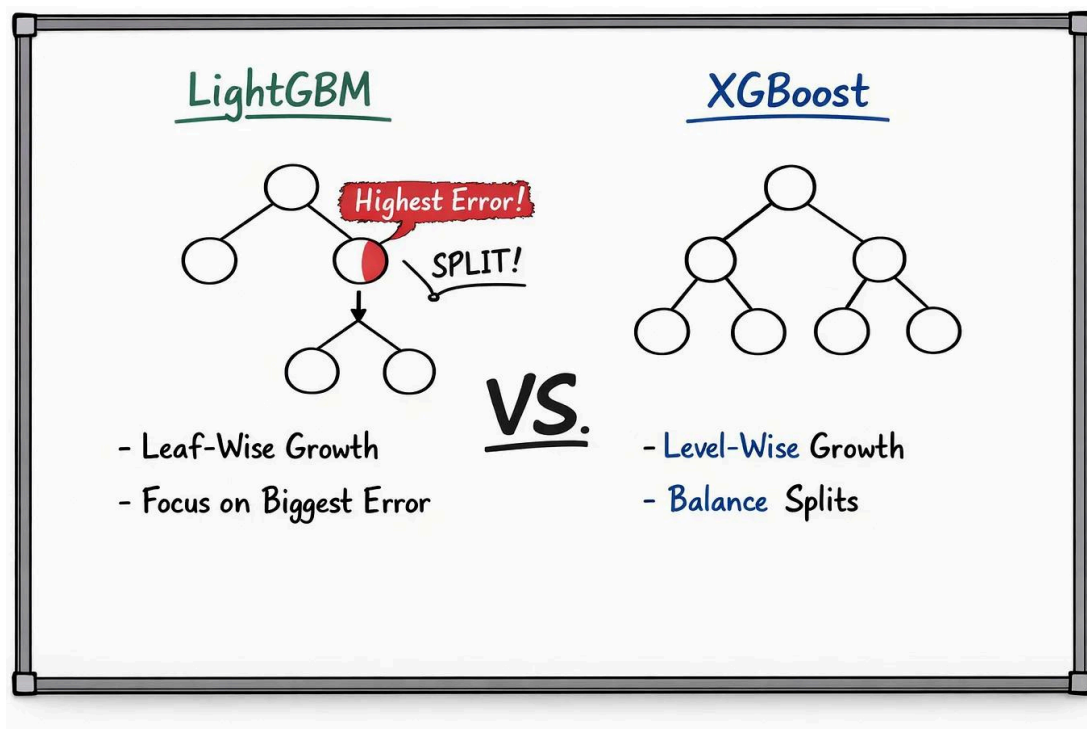
Hình 1.3. Thuật toán XGBoost

- Khái niệm: XGBoost (Extreme Gradient Boosting) là thuật toán học máy thuộc nhóm Gradient Boosting. Đây là một trong những thuật toán có hiệu năng cao nhất đối với dữ liệu dạng bảng và được sử dụng rộng rãi trong các cuộc thi khoa học dữ liệu. (*Gradient Boosting là phương pháp xây dựng nhiều mô hình theo trình tự, trong đó mỗi mô hình mới được tạo ra để sửa lỗi cho mô hình trước đó*)
 - + Ví dụ:
 1. Cây 1 dự đoán sai 30 sinh viên.
 2. Cây 2 tập trung học 30 trường hợp sai đó.
 3. Cây 3 tiếp tục sửa các lỗi còn lại.
 - + Sai số sẽ giảm dần qua từng vòng huấn luyện.)
- Nguyên lý hoạt động: Khác với Random Forest, các cây trong XGBoost được xây dựng tuần tự. Mỗi cây mới được tạo ra nhằm sửa lỗi cho các cây trước đó bằng cách tập trung học các mẫu dữ liệu dự đoán sai. Quá trình này giúp giảm dần sai số qua từng vòng huấn luyện và cải thiện khả năng dự đoán của mô hình.

- Ưu điểm:
 - + **Hiệu năng dự đoán cao:** Nhờ liên tục sửa lỗi qua từng vòng lặp, XGBoost thường đạt độ chính xác cao hơn nhiều thuật toán truyền thống.
 - + **Xử lý dữ liệu thiếu hiệu quả:** Thuật toán có cơ chế xử lý Missing Values mà không cần loại bỏ toàn bộ bản ghi.
 - + **Regularization giúp giảm Overfitting:** XGBoost bổ sung cơ chế Regularization nhằm kiểm soát độ phức tạp của mô hình và tăng khả năng tổng quát hóa. *(Regularization là cơ chế kiểm soát độ phức tạp của mô hình nhằm hạn chế hiện tượng Overfitting. Nếu mô hình tạo quá nhiều nhánh hoặc quá sâu, thuật toán sẽ áp dụng hình phạt để giữ cho mô hình đơn giản hơn và có khả năng tổng quát hóa tốt hơn.)*
- Nhược điểm:
 - + **Nhiều siêu tham số cần tinh chỉnh:** Để đạt kết quả tối ưu cần thử nghiệm nhiều bộ tham số khác nhau.
 - + **Thời gian huấn luyện tương đối lớn:** Quá trình học tuần tự khiến thời gian huấn luyện dài hơn Logistic Regression hoặc Random Forest.
- Vai trò trong đề tài: XGBoost được lựa chọn như một mô hình mạnh để đánh giá khả năng phát hiện sớm sinh viên có nguy cơ học tập thấp hoặc bỏ học.

1.4.4. LightGBM

- Khái niệm: LightGBM (Light Gradient Boosting Machine) là thuật toán học máy được phát triển bởi Microsoft dựa trên kỹ thuật Gradient Boosting. Thuật toán được thiết kế nhằm tăng tốc độ huấn luyện và giảm mức sử dụng bộ nhớ.
- Nguyên lý hoạt động: LightGBM sử dụng chiến lược phát triển cây theo lá (Leaf-wise Growth). Thay vì mở rộng đồng đều tất cả các nhánh, thuật toán chỉ tập trung mở rộng nhánh giúp giảm sai số nhiều nhất.



Hình 1.4. Thuật toán LightGBM

Bên cạnh đó, LightGBM sử dụng kỹ thuật Histogram-based để nhóm dữ liệu thành các khoảng giá trị nhằm giảm khối lượng tính toán.

- Ưu điểm:
 - + **Tốc độ huấn luyện cao:** LightGBM thường nhanh hơn XGBoost trên các tập dữ liệu lớn.
 - + **Tiết kiệm bộ nhớ:** Kỹ thuật Histogram giúp giảm đáng kể lượng RAM cần sử dụng.
 - + **Độ chính xác cao:** Khả năng tập trung vào các vùng dữ liệu quan trọng giúp mô hình đạt hiệu quả dự đoán tốt.
- Nhược điểm:
 - + **Dễ Overfitting trên dữ liệu nhỏ:** Chiến lược Leaf-wise có thể làm cây phát triển quá sâu nếu không được kiểm soát.
 - + **Khó giải thích:** Quá trình ra quyết định của mô hình phức tạp hơn các mô hình tuyến tính.
- Vai trò trong đề tài: LightGBM là mô hình trọng tâm của nghiên cứu và được kỳ vọng đạt hiệu năng tốt nhất trong bài toán phân loại mức độ rủi ro sinh viên.

1.5. Thang đo độ chính xác, hiệu suất và hiệu quả của mô hình máy học

Để đánh giá toàn diện mô hình phân loại đa lớp với dữ liệu mất cân bằng, đề tài sử dụng ba nhóm chỉ số:

- **F1-score (Macro):** Trung bình F1 trên tất cả các lớp với trọng số bằng nhau, phù hợp khi cần đánh giá đồng đều cả lớp thiểu số và đa số.
- **AUC-ROC (One-vs-Rest):** Diện tích dưới đường cong ROC theo chiến lược One-vs-Rest, đo khả năng phân biệt từng mức rủi ro so với phần còn lại.
- **Confusion Matrix:** Ma trận nhầm lẫn giúp phân tích chi tiết các trường hợp phân loại sai giữa các mức rủi ro, đặc biệt quan trọng với lớp High (thiểu số).

CHƯƠNG 2: PHƯƠNG PHÁP NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH MÁY HỌC

2.1. Xác định bài toán và dữ liệu

2.1.1. Phát biểu bài toán bài toán

Bài toán được mô hình hóa thành bài toán Phân loại có giám sát đa lớp (Multi-class Supervised Classification) với định nghĩa chính thức:

- Tập dữ liệu: $S = \{s_1, s_2, ..., s_n\}$ gồm $n = 32.593$ lượt đăng ký môn học.
- Thời điểm cắt (Time cut-off T): $T =$ ngày thứ 60 của khóa học, đảm bảo chỉ dùng dữ liệu có thể quan sát được tại thời điểm cảnh báo sớm.
- Vector đặc trưng X_i : Gồm thông tin tĩnh (nhân khẩu học) và thông tin động (tương tác VLE, điểm số) tính đến ngày T.
- Nhãn mục tiêu Y_i : $Y \in \{0, 1, 2, 3\}$ tương ứng Thấp, Trung bình, Cao, Rất cao.
- Xây dựng hàm $f: X \rightarrow Y$ tối ưu hóa Macro F1-score, đặc biệt giảm thiểu bỏ sót sinh viên nhóm rủi ro Cao và Rất cao.

2.1.2. Nguồn dữ liệu (Data Source)

Nghiên cứu sử dụng tập dữ liệu mã nguồn mở Open University Learning Analytics Dataset (OULAD) (Kuzilek et al., 2017). Đây là một trong những tập dữ liệu quy mô và toàn diện nhất về hành vi học tập trực tuyến, bao gồm dữ liệu nhân khẩu học, thông tin đăng ký, kết quả đánh giá và nhật ký tương tác (clickstream) trên hệ thống Virtual Learning Environment (VLE) của Đại học Mở Vương quốc Anh.

Bộ dữ liệu bao gồm 32.593 bản ghi sinh viên tham gia 22 module khóa học, tổ chức dưới dạng 7 bảng CSV. Nghiên cứu tập trung vào 4 bảng cốt lõi:

Bảng	Quy mô	Nội dung chính
studentInfo	32.593 bản ghi, 12 thuộc tính	Thông tin nhân khẩu học, kết quả cuối kỳ (final_result)
studentVle	>10.6 triệu bản ghi, 6 thuộc tính	Lịch sử tương tác hàng ngày trên LMS (date, sum_click)
assessments	206 bản ghi, 6 thuộc tính	Cấu trúc bài kiểm tra, hạn chót nộp bài (date)

studentAssessment	173.912 bản ghi, 5 thuộc tính	Lịch sử làm bài: điểm số, ngày nộp thực tế
-------------------	-------------------------------	--

- Kiểm tra các bản ghi và thuộc tính từng bảng:

```
# -- Load tất cả bảng --
student_info = pd.read_csv(RAW_DIR + 'studentInfo.csv')
student_asmt = pd.read_csv(RAW_DIR + 'studentAssessment.csv')
assessments = pd.read_csv(RAW_DIR + 'assessments.csv')
student_vle = pd.read_csv(RAW_DIR + 'studentVle.csv')
student_reg = pd.read_csv(RAW_DIR + 'studentRegistration.csv')
vle = pd.read_csv(RAW_DIR + 'vle.csv')

tables = {
    'studentInfo': student_info,
    'studentAssessment': student_asmt,
    'assessments': assessments,
    'studentVle': student_vle,
    'studentRegistration': student_reg,
    'vle': vle
}

for name, df in tables.items():
    print(f'{name:25s} | shape: {str(df.shape):15s} | cols: {list(df.columns)}')

Python
studentInfo      | shape: (32593, 12) | cols: ['code_module', 'code_presentation', 'id_student', 'gender', 'region', 'highest_education', 'imd_band', 'age_band', 'num_of_prev
studentAssessment | shape: (173912, 5) | cols: ['id_assessment', 'id_student', 'date_submitted', 'is_banked', 'score']
assessments       | shape: (286, 6) | cols: ['code_module', 'code_presentation', 'id_assessment', 'assessment_type', 'date', 'weight']
studentVle        | shape: (10655288, 6) | cols: ['code_module', 'code_presentation', 'id_student', 'id_site', 'date', 'sum_click']
studentRegistration | shape: (32593, 5) | cols: ['code_module', 'code_presentation', 'id_student', 'date_registration', 'date_unregistration']
vle               | shape: (6364, 6) | cols: ['id_site', 'code_module', 'code_presentation', 'activity_type', 'week_from', 'week_to']
```

2.2. Xây dựng pipeline xử lý dữ liệu

2.2.1. Làm sạch dữ liệu

Trước khi tiến hành làm sạch, nhóm có thực hiện rà soát các giá trị bị mất (Missing Value), ở các bảng để dễ dàng tiến hành xử lý mất mát từ dữ liệu gốc:

```
# -- Kiểm tra missing values toàn bộ bảng --
print('=== Missing Values Summary ===')
for name, df in tables.items():
    missing = df.isnull().sum()
    if missing.any():
        print(f'\n[{name}]')
        print(missing[missing > 0].to_string())
    else:
        print(f'[{name}] Không có missing')
```

Bảng	Thuộc tính	Số lượng Missing value
student info	imd_band	1111
studentAssessment	score	173
assessments	date	11
studentVle		Không có Missing
studentRegistration	date_registration	45
	date_unregistration	22521

vle	week_from	5243
	week_to	5243

2.2.1.1. Làm sạch bảng *StudentInfo*

- Dữ liệu thô được xử lý qua quy trình kiểm soát chất lượng nghiêm ngặt.
- Xây dựng bảng ánh xạ chuyển đổi các dải phần trăm sang thang điểm 1–10. Giá trị thiếu được điền bằng median để duy trì tính ổn định phân phối.

```
# Mapping imd_band (Index of Multiple Deprivation) sang số thứ tự
imd_order = {
    '0-10%': 1, '10-20%': 2, '20-30%': 3, '30-40%': 4, '40-50%': 5,
    '50-60%': 6, '60-70%': 7, '70-80%': 8, '80-90%': 9, '90-100%': 10
}
```

- Điền missing imd_band bằng encoding, tương tự với các thuộc tính education, age, gender, disability...

```
edu_order = {
    'No Formal quals': 0,
    'Lower Than A Level': 1,
    'A Level or Equivalent': 2,
    'HE Qualification': 3,
    'Post Graduate Qualification': 4
}
age_order = {'0-35': 0, '35-55': 1, '55<=': 2}

si = student_info.copy()

# Điền missing imd_band bằng median encoding
si['imd_band_num'] = si['imd_band'].map(imd_order)
si['imd_band_num'].fillna(si['imd_band_num'].median(), inplace=True)

# Ordinal encoding
si['education_num'] = si['highest_education'].map(edu_order)
si['age_num'] = si['age_band'].map(age_order)

# Binary encoding
si['gender_num'] = (si['gender'] == 'M').astype(int)
si['disability_num'] = (si['disability'] == 'Y').astype(int)

print('Sau khi làm sạch:', si.shape)
print('\nCác cột mới:', ['imd_band_num', 'education_num', 'age_num', 'gender_num', 'disability_num'])
```

Trước khi làm sạch: (32593, 12)

Sau khi làm sạch: (32593, 17)

Các cột mới: ['imd_band_num', 'education_num', 'age_num', 'gender_num', 'disability_num']

- Sau khi làm sạch giá trị missing và encoding ta thu được bộ dữ liệu sạch gồm 32593 bản ghi, thêm 3 cột lưu trữ giá trị encoding phục vụ cho huấn luyện máy học.

2.2.1.2. Xử lý missing score bằng StudentAssessment

- Tìm ra số bản ghi bị mất dữ liệu, những dữ liệu bị mất (Score = null) thì coi như sinh viên đã không nộp bài => Score = 0.

```
# — 3.2 studentAssessment – xử lý missing score và is_banked —————
sa = student_asmt.copy()

print(f'Tổng bản ghi assessment: {len(sa):,}')
print(f'Missing score: {sa["score"].isnull().sum():,} ({sa["score"].isnull().mean()*100:.1f}%)')

# Score null = không nộp bài → xem như 0
sa['score'].fillna(0, inplace=True)
sa['submitted'] = (~student_asmt['score'].isnull()).astype(int) # flag nộp hay không

# Loại bài "banked" từ kỳ trước (tránh data leakage)
sa = sa[sa['is_banked'] == 0].copy()
print(f'Sau khi loại banked: {len(sa):,}')
```

Tổng bản ghi assessment: 173,912
Missing score: 173 (0.1%)
Sau khi loại banked: 172,003

- Sau khi làm sạch giá trị missing thu được 172,003 bản ghi.

2.2.1.3. Xử lý bằng studentRegistration

- Tính toán các ngày đăng ký sớm/muộn hoặc đã hủy đăng ký, số lượng sinh viên, tỉ lệ phần trăm ở các mốc 25% 50% 75%.
- Phần trăm thể hiện từng từ ngưỡng mức đó trở xuống, đánh giá số lượng sinh viên ở từng mốc.

```
# — 3.3 studentRegistration – tính ngày đăng ký sớm/muộn —————
reg = student_reg.copy()

# Ngày đăng ký âm = đăng ký trước khi khóa bắt đầu (tốt)
reg['early_registration'] = (reg['date_registration'] < 0).astype(int)
reg['reg_days_before'] = reg['date_registration'].clip(upper=0).abs() # bao nhiêu ngày trước

# Đã hủy đăng ký chưa
reg['unregistered'] = (~reg['date_unregistration'].isnull()).astype(int)

print(reg[['early_registration', 'reg_days_before', 'unregistered']].describe())
```

	early_registration	reg_days_before	unregistered
count	32593.00	32548.00	32593.00
mean	0.99	69.50	0.31
std	0.09	49.10	0.46
min	0.00	0.00	0.00
25%	1.00	29.00	0.00
50%	1.00	57.00	0.00
75%	1.00	100.00	1.00
max	1.00	322.00	1.00

2.2.2. Mã hóa biến

- Mã hóa thứ tự (Ordinal Encoding): Áp dụng cho education_num (0–4), age_num (0–2), imd_band_num (1–10) để bảo toàn mối quan hệ thứ bậc tự nhiên.

```
# Mapping imd_band (Index of Multiple Deprivation) sang số thứ tự
imd_order = {
    '0-10%': 1, '10-20%': 2, '20-30%': 3, '30-40%': 4, '40-50%': 5,
    '50-60%': 6, '60-70%': 7, '70-80%': 8, '80-90%': 9, '90-100%': 10
}
edu_order = {
    'No Formal quals': 0,
    'Lower Than A Level': 1,
    'A Level or Equivalent': 2,
    'HE Qualification': 3,
    'Post Graduate Qualification': 4
}
age_order = {'0-35': 0, '35-55': 1, '55<=': 2}
```

- Mã hóa nhị phân (Binary Encoding): Áp dụng cho gender_num (Nam=1, Nữ=0), disability_num (Có=1, Không=0), early_registration

```
# Binary encoding
si['gender_num'] = (si['gender'] == 'M').astype(int)
si['disability_num'] = (si['disability'] == 'Y').astype(int)
```

=> Mã hóa biến giúp việc huấn luyện trở nên nhanh chóng và dễ dàng hơn.

2.2.3. Feature Engineering

- Tạo Feature từ studentVle theo sinh viên kết hợp module và presentation:
 - + total_clicks: Tổng số lượt click
 - + active_days: Số ngày có tương tác
 - + avg_clicks_day: Click trung bình/ngày
 - + max_clicks_day: Ngày có số lượt click nhiều nhất
 - + n_resources: Số tài nguyên khác nhau đã xem

```

# — Aggregate studentVle theo sinh viên + module + presentation —————
vle_features = student_vle.groupby(
    ['id_student', 'code_module', 'code_presentation']
).agg(
    total_clicks = ('sum_click', 'sum'),          # tổng lượt click
    active_days   = ('date', 'nunique'),          # số ngày cố tương tác
    avg_clicks_day = ('sum_click', 'mean'),       # click trung bình/ngày
    max_clicks_day = ('sum_click', 'max'),        # ngày bận nhất
    n_resources   = ('id_site', 'nunique'),       # số tài nguyên khác nhau đã xem
).reset_index()

# Click density = tổng click / số ngày hoạt động
vle_features['click_density'] = (
    vle_features['total_clicks'] / vle_features['active_days'].clip(lower=1)
)

print(f'VLE features shape: {vle_features.shape}')
display(vle_features.describe())

```

- Features từ Assessment (điểm và nộp bài:

```

# — Join assessment với metadata để phân loại TMA/CMA/Exam —————
sa_full = sa.merge(assessments, on='id_assessment', how='left')

# Tính submission delay (nộp sớm/muộn so với deadline)
sa_full['submit_delay'] = sa_full['date_submitted'] - sa_full['date']
# Âm = nộp trước deadline, Dương = nộp muộn
sa_full['late_submission'] = (sa_full['submit_delay'] > 0).astype(int)

# Aggregate theo sinh viên + module + presentation
asmt_features = sa_full.groupby(
    ['id_student', 'code_module', 'code_presentation']
).agg(
    avg_score      = ('score', 'mean'),
    min_score      = ('score', 'min'),
    std_score      = ('score', 'std'),
    n_submitted    = ('submitted', 'sum'),
    n_late         = ('late_submission', 'sum'),
    avg_submit_delay = ('submit_delay', 'mean'),
).reset_index()

# TMA score riêng (bài giáo viên chấm)
tma_scores = sa_full[sa_full['assessment_type'] == 'TMA'].groupby(
    ['id_student', 'code_module', 'code_presentation']
).agg(avg_tma_score=('score', 'mean')).reset_index()

asmt_features = asmt_features.merge(tma_scores, on=['id_student', 'code_module', 'code_presentation'], how='left')
asmt_features['avg_tma_score'].fillna(asmt_features['avg_score'], inplace=True)
asmt_features['std_score'].fillna(0, inplace=True)

print(f'Assessment features shape: {asmt_features.shape}')
display(asmt_features.describe())

```

- + Join Metadata: Kết hợp bảng sa (studentAssessment) với assessments là để lấy được cột date (hạn chót nộp bài) và assessment_type (loại bài kiểm tra).
- + Tính toán Submission Delay: $\text{submit_delay} = \text{date_submitted} - \text{date}$. Sinh viên thường xuyên nộp muộn (delay > 0) hoặc nộp sát nút (delay $\approx$ 0) thường có rủi ro bỏ học hoặc trượt cao hơn những người nộp sớm.
- + Xử lý TMA Score riêng biệt: Code tách riêng avg_tma_score (Tutor Marked Assessment - bài tập do giáo viên chấm).

Hệ thống tổng hợp 23 đặc trưng từ 4 nhóm:

Nhóm thuộc tính	Bảng nguồn	Tên thuộc tính	Mô tả
Nhân khẩu học	studentInfo	gender_num	Chuyển đổi giới tính (1: Nam, 0: Nữ)
		imd_band_num	Ordinal encoding mức độ nghèo đói (1-10)
		education_num	Ordinal encoding trình độ học vấn (0-4)
		age_num	Ordinal encoding nhóm tuổi (0-2)
		disability_num	Trạng thái khuyết tật (1: Có, 0: Không)
		num_of_prev_attempts	Số lần học lại môn này trước đó
		studied_credits	Tổng số tín chỉ đang theo học
Đăng ký học	studentRegistration	early_registration	1 nếu đăng ký trước khi khóa học bắt đầu
		reg_days_before	Số ngày đăng ký trước khi khai giảng (trị tuyệt đối)
		unregistered	1 nếu sinh viên đã hủy đăng ký (Withdrawn)
Tương tác VLE	studentVle	total_clicks	Tổng số lượt click vào tài liệu học tập

		active_days	Tổng số ngày có tương tác với hệ thống
		avg_clicks_day	Số lượt click trung bình mỗi ngày
		max_clicks_day	Số lượt click lớn nhất trong một ngày
		n_resources	Số lượng tài nguyên (link/file) khác nhau đã xem
		click_density	Mật độ tương tác (Tổng click / Số ngày hoạt động)
Điểm & Nộp bài	studentAssessment & assessments	avg_score	Điểm trung bình tất cả các bài kiểm tra
		min_score	Điểm thấp nhất trong các bài đã nộp
		std_score	Độ lệch chuẩn của điểm số
		avg_tma_score	Điểm trung bình các bài do giảng viên chấm (TMA)
		n_submitted	Tổng số bài kiểm tra đã nộp
		n_late	Số bài nộp muộn sau deadline

		avg_submit_delay	Thời gian nộp bài trung bình so với deadline (âm là sớm)
--	--	------------------	--

2.2.4. Kết nối bảng

Dữ liệu từ bốn bảng studentInfo, studentRegistration, VLE features và Assessment features được kết nối bằng phương pháp LEFT JOIN theo khóa chung [id_student, code_module, code_presentation], đảm bảo không mất dữ liệu của bất kỳ sinh viên nào trong tập chính.

```
# — Chọn cột cần thiết từ studentInfo —————
❖ si_cols = [
    'id_student', 'code_module', 'code_presentation',
    'gender_num', 'imd_band_num', 'education_num',
    'age_num', 'disability_num',
    'num_of_prev_attempts', 'studied_credits',
    'final_result'
]
base = si[si_cols].copy()

# — Chọn cột registration —————
❖ reg_cols = [
    'id_student', 'code_module', 'code_presentation',
    'early_registration', 'reg_days_before', 'unregistered'
]

# — JOIN tuần tự —————
join_keys = ['id_student', 'code_module', 'code_presentation']

df = base.merge(reg[reg_cols], on=join_keys, how='left')
df = df.merge(vle_features, on=join_keys, how='left')
df = df.merge(asmt_features, on=join_keys, how='left')

print(f' DataFrame sau JOIN: {df.shape}')
print(f' Số sinh viên: {df["id_student"].nunique():,}')
print(f'\n=== Missing Values sau JOIN ===')
missing_final = df.isnull().sum()
print(missing_final[missing_final > 0])
```

- Thống kê dataframe sau JOIN: (32593, 27).
- Số sinh viên: 28,785.
- Tuy nhiên việc kết nối theo LEFT JOIN sẽ gây ra trường hợp Missing Value:

Thuộc tính	Missing Value sau JOIN
imd_band_num	1111
reg_days_before	45
total_clicks	3365
active_days	3365
avg_clicks_day	3365
max_clicks_day	3365
n_resources	3365

click_density	3365
avg_score	7036
min_score	7036
std_score	9580
n_submitted	7013
n_late	7013
avg_submit_delay	7024
avg_tma_score	8097

- Nguyên nhân:
 - + Trong quá trình học tập có những sinh viên chưa từng đăng nhập hoặc tương tác với hệ thống học tập trực tuyến (Nhóm Vle total_clicks, active_days...). Những sinh viên không có tên trong bảng Vle thì sẽ bị gán NaN khi dùng LEFT JOIN
 - + Có những sinh viên chưa từng nộp bất kì bài kiểm tra nào. Tương tự họ không tồn tại trong bảng studentAssessment. Khoảng 7,000 sinh viên này có thể là những người đã hủy đăng ký ngay từ đầu hoặc bỏ học trước khi kỳ thi đầu tiên diễn ra.
 - + avg_tma_score: cao hơn 8097 missing vì có một số sinh viên có nộp bài máy tính (CMA) nhưng không nộp bài thi giáo viên chấm (TMA).
 - + imd_band_num (1,111 missing): Đây là các lỗi dữ liệu gốc từ bảng studentInfo. Một số sinh viên không được ghi nhận chỉ số nghèo đói (IMD) tại khu vực họ sống.
 - + reg_days_before (45 missing): Có 45 trường hợp sinh viên không có ngày đăng ký cụ thể trong hệ thống.
- Cách giải quyết, sau JOIN nhóm thực hiện điền missing cho các giá trị bị NaN, những trường hợp đó sẽ được gán = 0:

```

# — Điền missing sau JOIN (sv chưa tương tác VLE hoặc chưa nộp bài) —————
vle_fill_cols = ['total_clicks', 'active_days', 'avg_clicks_day', 'max_clicks_day',
                 'n_resources', 'click_density']
asmt_fill_cols = ['avg_score', 'min_score', 'std_score', 'n_submitted', 'n_late',
                 'avg_submit_delay', 'avg_tma_score']
reg_fill_cols = ['early_registration', 'reg_days_before', 'unregistered']

# VLE null = không tương tác → 0
df[vle_fill_cols] = df[vle_fill_cols].fillna(0)
# Assessment null = chưa nộp bài → score 0
df[asmt_fill_cols] = df[asmt_fill_cols].fillna(0)
# Registration null → 0
df[reg_fill_cols] = df[reg_fill_cols].fillna(0)

print('Missing values sau xử lý:', df.isnull().sum().sum())
print(f'Shape cuối: {df.shape}')

```

```

Missing values sau xử lý: 1111
Shape cuối: (32593, 27)

```

2.2.5. Gán nhãn mức rủi ro

Hệ thống xây dựng nhãn mục tiêu 4 mức dựa trên logic tổng hợp kết quả học tập và hành vi:

Mức	Nhãn	Điều kiện
3	Very High	final_result = Fail hoặc Withdraw
2	High	Pass nhưng điểm rất thấp hoặc không có tương tác hệ thống trực tuyến
1	Medium	Pass nhưng điểm biên hoặc TMA yếu hoặc nhiều bài nộp muộn
0	Low	Pass

```
def assign_risk_label(row):
    result = row['final_result']
    score = row['avg_score']
    clicks = row['total_clicks']
    tma = row['avg_tma_score']
    late = row['n_late']
    active = row['active_days']

    # — Mức 3: Rất cao —————
    if result in ['Fail', 'Withdrawn']:
        return 3

    # — Mức 2: Cao —————
    # Pass nhưng điểm rất thấp hoặc gần như không tương tác VLE
    if score < 55 or clicks < 100 or active < 5:
        return 2

    # — Mức 1: Trung bình —————
    # Pass nhưng điểm biên hoặc TMA yếu hoặc nhiều bài nộp muộn
    if score < 70 or tma < 65 or (late >= 2 and score < 75):
        return 1

    # — Mức 0: Thấp —————
    return 0
```

- Kết quả phân phối nhãn cuối cùng được tổng hợp dưới bảng sau:

risk_label	Count	Percentage (%)
Very High	17208	52,80
High	10724	32,90
Medium	4048	12,40
Low	613	1,90

2.2.6. Train/ Validation/ Test split

Sử dụng kĩ thuật Stratified Split (Phân chia tầng): Dữ liệu rủi ro của sinh viên là không đồng đều. Nếu lấy ngẫu nhiên thông thường, tập Test có thể tình cờ không chứa sinh viên rủi ro High, dẫn đến việc đánh giá mất đi ý nghĩa thực tiễn. Kỹ thuật chia phân tầng đảm bảo rằng dù chia nhỏ ra sao, tỷ lệ phần trăm các lớp Low, Medium, High, Very High ở cả ba tập Train, Valid, Test đều tỉ lệ thuận tuyệt đối với bức tranh toàn cảnh ban đầu

```
# 2. Thực hiện chia tập dữ liệu 60/20/20 với tham số stratify chống suy biến dữ liệu
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42)

print(f"\nKích thước tập Train: {X_train.shape}")
print(f"\nKích thước tập Validation: {X_val.shape}")
print(f"\nKích thước tập Test: {X_test.shape}")
```

2.2.7. Chuẩn hóa dữ liệu

Dữ liệu sinh viên bao gồm các đặc trưng có thang đo (scale) chênh lệch rất lớn. Đặc trưng tần suất tương tác (total_clicks) dao động từ 0 đến hàng chục nghìn lượt, trong khi điểm bình quân (avg_score) chỉ đạt tối đa là 100.

Sự chênh lệch này khiến các thuật toán tối ưu hóa (như Gradient Descent) hoạt động thiếu hiệu quả do bị "thiên vị", nhằm tưởng các biến số có giá trị lớn là quan trọng hơn. Nhóm sử dụng StandardScaler để chuyển dịch toàn bộ dữ liệu về phân phối chuẩn (Standard Normal Distribution):

- Trung bình (μ) = 0
- Độ lệch chuẩn (σ) = 1

```
# 3. Chuẩn hóa dữ liệu StandardScaler và lưu vào Models để triển khai API sau này
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
joblib.dump(scaler, os.path.join(models_dir, 'scaler.pkl'))
```

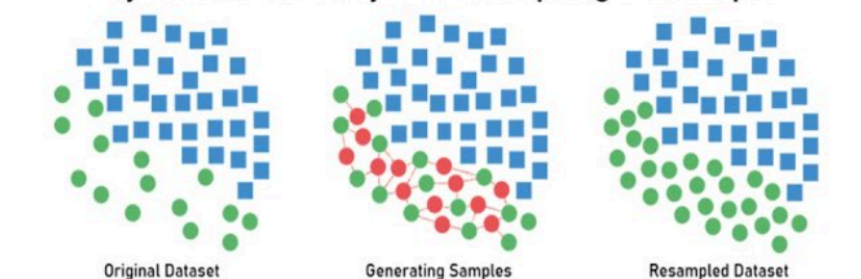
2.3. Xử lý dữ liệu mất cân bằng

Trong dữ liệu giáo dục, phần đông sinh viên có tình trạng học tập bình thường, trong khi tệp sinh viên bỏ học hoặc có rủi ro học vụ cao (High / Very High) chiếm số lượng rất nhỏ.

Sự thiên lệch này gây ảnh hưởng nghiêm trọng đến các mô hình phân loại. Mô hình dễ học được một "mẹo" tiêu cực: chỉ cần dự đoán tất cả học sinh thuộc nhóm rủi ro "Low" thì độ chính xác (Accuracy) vẫn đạt trên 80%, nhưng hệ thống Cảnh báo Sớm (EWS) sẽ bỏ lọt 100% lứa sinh viên có nguy cơ thực sự. Để giải quyết vấn đề này, dự án áp dụng giải pháp

SMOTE – Imbalanced Data

Synthetic Minority Oversampling Technique



Thuật toán SMOTE (Synthetic Minority Over-sampling Technique): Thay vì sao chép dữ liệu cũ (dễ gây Overfitting), SMOTE sử dụng thuật toán k-Nearest Neighbors. Nó chọn một mẫu thuộc lớp thiểu số, đo các điểm lân cận trong không gian đa chiều, sau đó nội suy toán học để "vẽ" thêm các điểm dữ liệu nhân tạo mới. Điều này làm phong phú tập huấn luyện cho nhóm High và Very High.

```
# 4. Xử lý mất cân bằng dữ liệu trên tập huấn luyện tạo môi trường công bằng
1:1:1:1
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)

print(f"\nPhân phối nhân sau thuật toán SMOTE:")
print(pd.Series(y_train_resampled).value_counts())
# Kịch bản Output hoàn hảo: Lớp Low, Medium, High, Very High được cân bằng ngang
bằng nhau
```

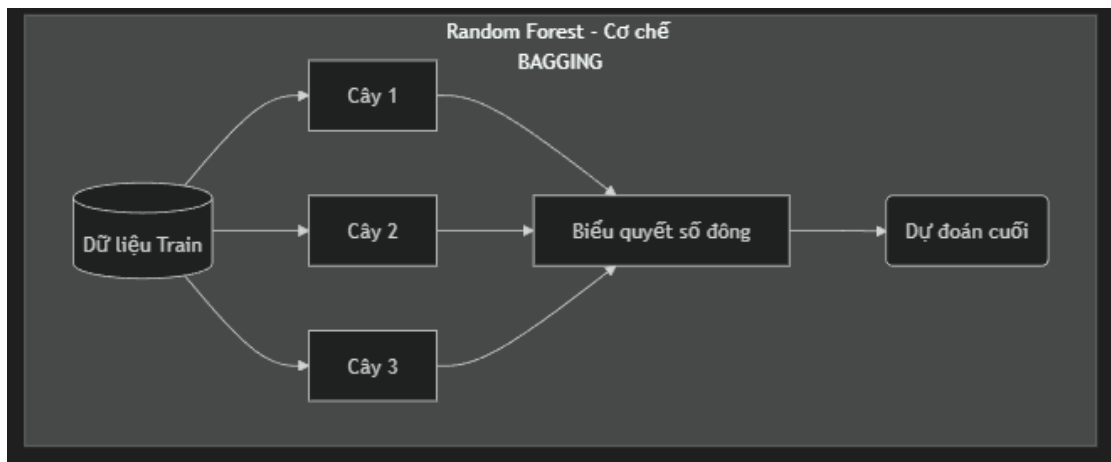
2.4. Mô hình máy học phù hợp

- Logistic Regression: Mô hình tuyến tính baseline, đơn giản và giải thích được, dùng để tạo ngưỡng so sánh tối thiểu.
- Random Forest: Ensemble học từ nhiều cây quyết định theo cơ chế bagging, ổn định với dữ liệu nhiễu.
- XGBoost: Gradient boosting tối ưu hóa theo từng bước, xử lý tốt missing values và đạt hiệu năng cao.
- LightGBM: Gradient boosting với kỹ thuật histogram-based, tốc độ huấn luyện nhanh, thường đạt AUC cao nhất trên dữ liệu lớn.

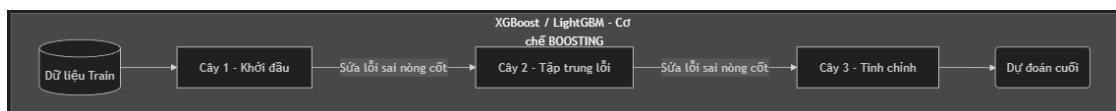
```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42,
class_weight='balanced'),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42,
class_weight='balanced'),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='mlogloss',
random_state=42),
    'LightGBM': LGBMClassifier(class_weight='balanced', random_state=42)
}
```

Logistic Regression (Baseline): Giải quyết bài toán bằng phương trình tuyến tính và hàm kích hoạt Multinomial Softmax. Dù minh bạch nhưng thường yếu thế khi dữ liệu có tương tác đa chiều phức tạp.

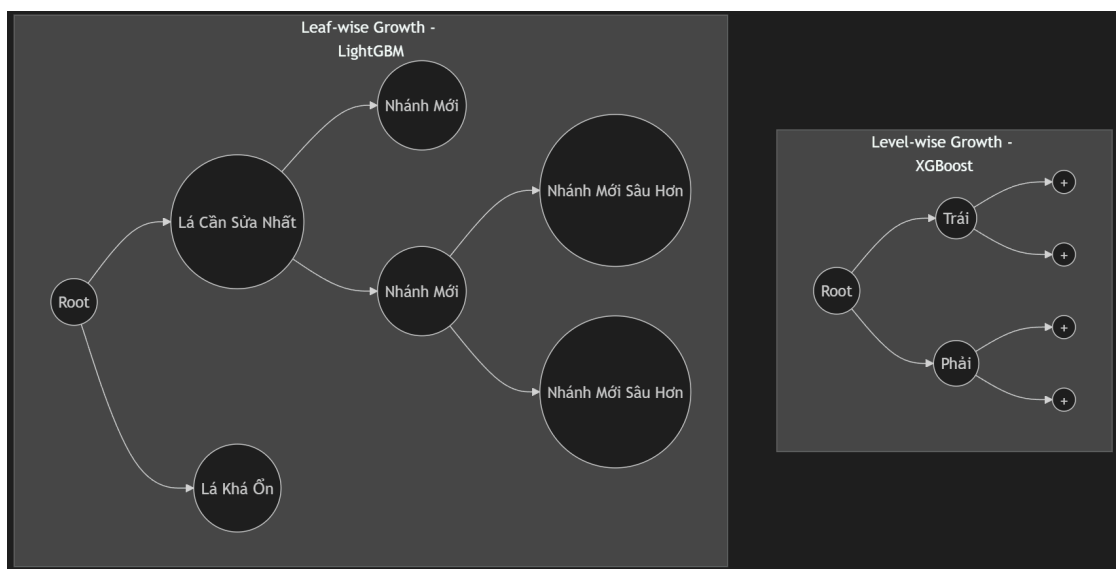
Random Forest: Sử dụng chiến thuật Bagging. Xây dựng độc lập hàng trăm cây quyết định, mỗi cây học một phần dữ liệu và biểu quyết kết quả cuối cùng, giúp triệt tiêu hiện tượng Overfitting.



XGBoost (Extreme Gradient Boosting): Các cây quyết định được học nối tiếp nhau. Cây sau tập trung sửa lỗi cho cây trước (Minimize Gradient Residuals), là thuật toán thống trị dữ liệu dạng bảng.



LightGBM (Mô hình trọng tâm): Cải tiến từ XGBoost với cơ chế Leaf-wise growth (phát triển cây theo chiều sâu tại lá có lỗi lớn nhất) và sử dụng biểu đồ Histogram để nén dữ liệu. LightGBM cho tốc độ huấn luyện nhanh gấp 5-10 lần và chiếm ít RAM hơn, cực kỳ phù hợp với dữ liệu tương tác VLE không lồ.



2.5. Phương pháp đánh giá

Bộ chỉ số đo lường được thiết kế đặc thù cho Hệ thống Cảnh báo Sớm (EWS):

1. **Macro F1-Score (Chỉ số công bằng):** Nhóm chọn Macro F1 làm chỉ số gốc thay vì Accuracy. Macro F1 tính trung bình cộng F1-Score của từng lớp một cách độc lập. Mô hình chỉ đạt điểm cao nếu nhận diện tốt cả những nhóm thiểu số (High/Very High).
2. **Ma trận nhầm lẫn (Confusion Matrix):** Sử dụng biểu đồ Heatmap để quan sát sự nhầm lẫn giữa các biên giới rủi ro (ví dụ: ranh giới mỏng manh giữa Medium và High). Đường chéo chính biểu thị các trường hợp dự đoán đúng tuyệt đối.
3. **AUC-ROC:** Với bài toán đa lớp, chiến lược OVR được áp dụng để đo lường diện tích đường cong (AUC), từ đó làm căn cứ đánh giá khả năng phân cấp ranh giới rủi ro của từng thuật toán.

CHƯƠNG 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ

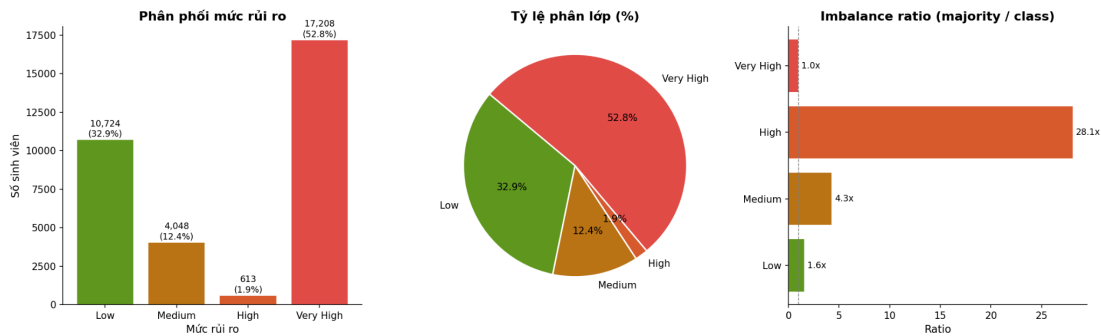
3.1. Môi trường thực nghiệm

Để đảm bảo tính nhất quán, khả năng tái lập của các thử nghiệm (reproducibility) cũng như tối ưu hóa tốc độ huấn luyện các mô hình học máy, môi trường phần cứng và phần mềm được thiết lập đồng nhất trong quá trình xây dựng hệ thống phân tầng rủi ro sinh viên. Thực nghiệm được thực hiện trên môi trường:

- Ngôn ngữ: Python 3.11
- Thư viện chính: Pandas, NumPy, Scikit-learn, XGBoost, LightGBM, Matplotlib, Seaborn
- Công cụ: VS Code
- Dữ liệu: Bộ dữ liệu OULAD đã được xử lý và trích xuất đặc trưng

3.2. Kết quả phân tích dữ liệu

Sau quá trình tiền xử lý, trích xuất đặc trưng và kết nối dữ liệu từ các file CSV rời rạc thành một bảng thống nhất, tiến hành chạy phân tích dữ liệu khám phá (EDA) kết hợp định danh nhãn rủi ro (Risk Labels). Các nhãn phân lớp rủi ro bao gồm: Mức 0 (Low), Mức 1 (Medium), Mức 2 (High), và Mức 3 (Very High).



Mức rủi ro	Số lượng	Tỷ lệ (%)	Imbalance ratio	Đánh giá
Low (Thấp)	10.724	32.9%	1.6x	Đa số
Medium (Trung bình)	4.048	12.4%	4.3x	Thiếu số trung bình
High (Cao)	613	1.9%	28.1x	Thiếu số nghiêm trọng

Very High (Rất cao)	17.208	52.8%	1.0x (đa số)	Fail + Withdrawn thực tế
---------------------	--------	-------	--------------	--------------------------

- + Nhóm Very High chiếm đa số bao gồm những sinh viên thực tế đã trượt hoặc bỏ học.
- + Nhóm Low chiếm 32.9% bên cạnh đa số sinh viên trượt hoặc bỏ học thì nhóm sinh viên học tốt cũng khá đông đảo.
- + Nhóm Medium & High lại chiếm khá thấp cho thấy tình hình học tập của sinh viên ở mức trung bình khá thấp có thể là do vài sinh viên học không vượt trội lắm so với đa số.

Chỉ số Imbalance Ratio (Tỷ lệ mất cân bằng): là một chỉ số thống kê dùng để đo lường mức độ chênh lệch giữa các lớp (classes) trong một bài toán phân loại của Machine Learning.

- Công thức tính: **IR** được tính bằng cách lấy số lượng mẫu của lớp chiếm đa số (Majority Class) chia cho số lượng mẫu của lớp thiểu số (Minority Class):
- Số lượng sinh viên lớp đa số (17,208 mẫu), lớp thiểu số (613 mẫu) thì chia ra $17,208/613 \sim 28,07$ nên ta thấy ở Nhóm High mất cân bằng đang diễn ra rất nghiêm trọng
- Các ngưỡng đánh giá thông thường:
 - + $IR = 1:1$: Tập dữ liệu cân bằng hoàn hảo (Balanced).
 - + $IR < 3:1$: Mất cân bằng nhẹ (Mild).
 - + IR từ 3:1 đến 10:1: Mất cân bằng trung bình (Moderate).
 - + $IR > 10:1$: Mất cân bằng nghiêm trọng (Extreme/Severe).

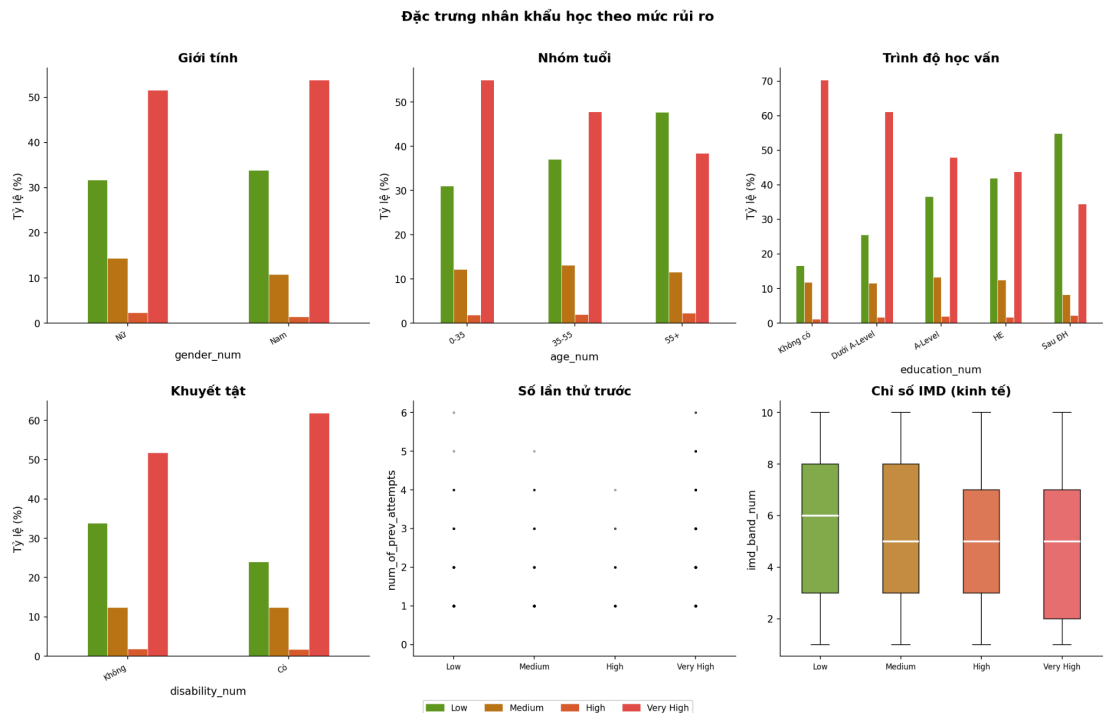
=> Với chỉ số như vậy khi mô hình AI học sẽ dễ bị đánh giá

3.2.1. Phân tích các yếu tố chi phối rủi ro sinh viên (EDA)

Thay vì trực tiếp đưa vào huấn luyện mô hình, bước khám phá này đóng vai trò quan trọng trong việc minh bạch hóa các đặc tính ẩn (Insights) của hệ thống dữ liệu

3.2.1.1. Đặc trưng dân số học (Demographics)

Sự chi phối của các yếu tố kinh tế - xã hội đối với khả năng bỏ học của sinh viên được thể hiện rõ ràng.



Hình 3.1. Biểu đồ đánh giá tương quan các đặc trưng nhân khẩu học

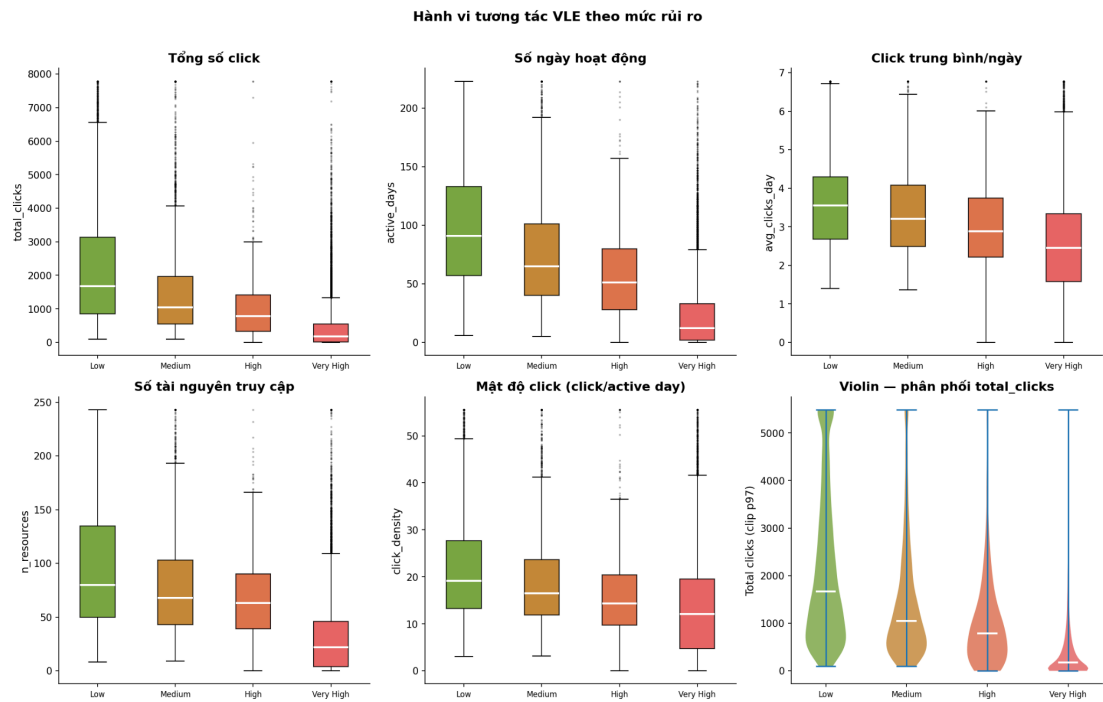
- Tỷ lệ nam nữ ở các mức rủi ro khá cân bằng nhau, với các mức đều có tỷ lệ tương đương
- Nhóm tuổi 55+ lại có mức Very High Risk thấp hơn theo đó Low lại cao hơn so với sinh viên từ 0 - 35 phán đoán là do người ở độ tuổi già có ý thức học hơn thế hệ trẻ, cũng một phần do môi trường.
- Với môi trường học tập OULAD thì trình độ học vấn có tỷ lệ Low và Very High đối ngược nhau, người có trình độ cao càng chú tâm học và ngược lại
- imd_band_num (Chỉ số kinh tế): Ở các khu vực nghèo nhất (0-10%), tỷ lệ sinh viên thuộc nhóm Very High Risk cao hơn hẳn các khu vực giàu có (90-100%).
- num_of_prev_attempts (Số lần học lại): Sinh viên học lại lần thứ 2, thứ 3 có xác suất rơi vào nhóm Very High Risk tăng vọt so với người học lần đầu.

3.2.1.2. Ảnh hưởng của nền tảng VLE (Mật độ Clicks)

Không thể phủ nhận mối quan hệ tuyến tính khá chặt chẽ giữa tổng số lượt click truy cập vào kho đồ án e-learning (vle) đối với học lực. Sinh viên duy trì được thói quen truy cập có tỷ lệ dính vào mức "Very High Risk" cực kỳ thấp.

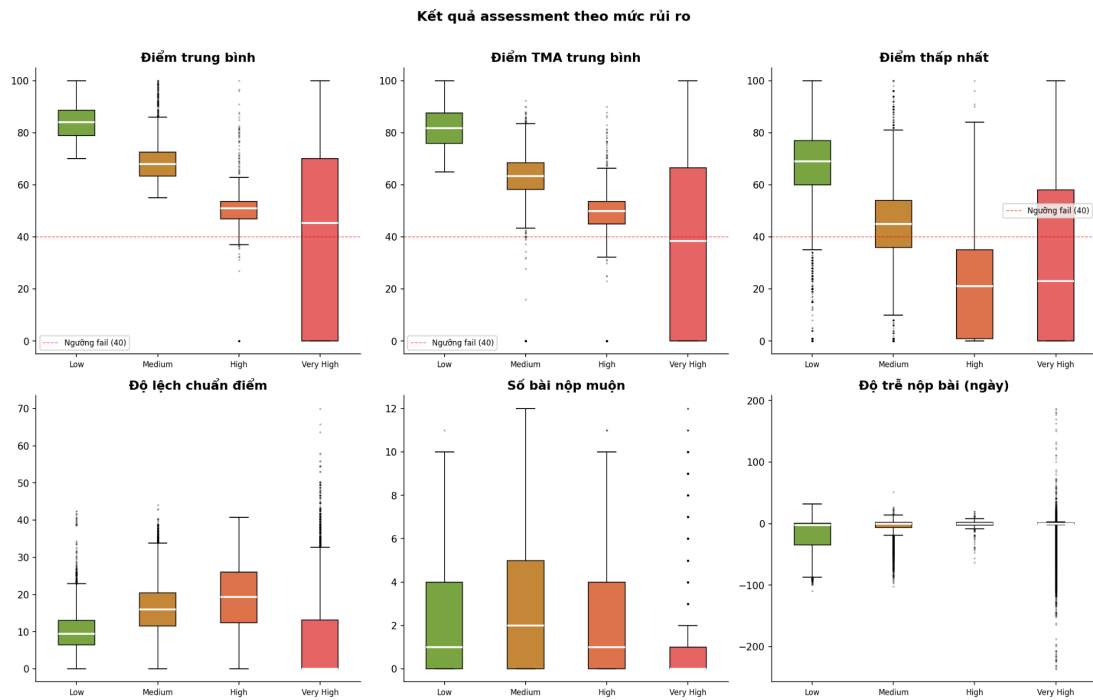
- Biểu đồ Boxplot (Hộp):

- + Đường kẻ trắng giữa hộp: Là giá trị Trung vị (Median).
- + Giá trị cụ thể (Median total_clicks): Nhóm Low Risk: 1,673 click. Nhóm Very High Risk: Chỉ 175 click (thấp hơn gần 10 lần).
- + Median active_days (Số ngày vào học): Nhóm Low học 91 ngày, nhóm Very High chỉ học 12 ngày.
- Ý nghĩa: Sự chăm chỉ (tần suất tương tác) là yếu tố phân biệt rõ rệt nhất giữa người đậu và người trượt. Sinh viên không vào hệ thống trong 1-2 tuần đầu có nguy cơ bỏ học cực cao.



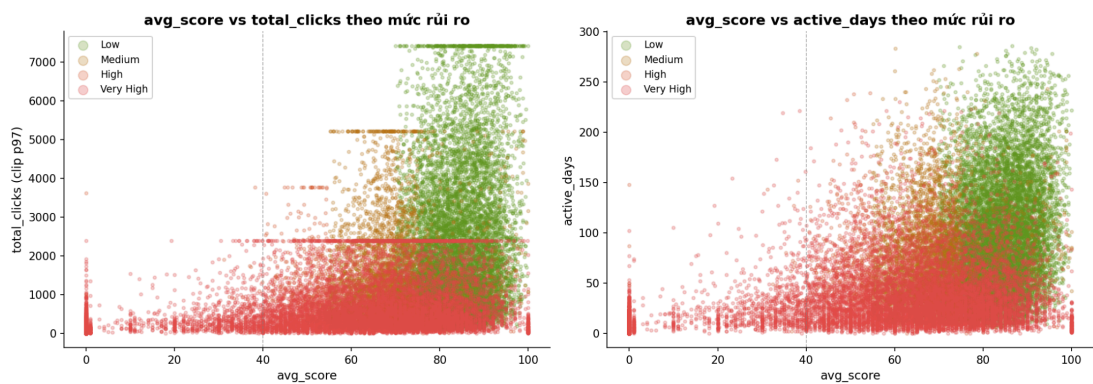
Hình 3.2. Tương quan sâu sắc của hành vi tương tác VLE đối với kết quả duy trì lớp học.

3.2.1.3. Kết quả Assessment theo mức độ rủi ro



Hình 3.3. Kết quả tình hình học tập phân theo hộp các ngưỡng rủi ro.

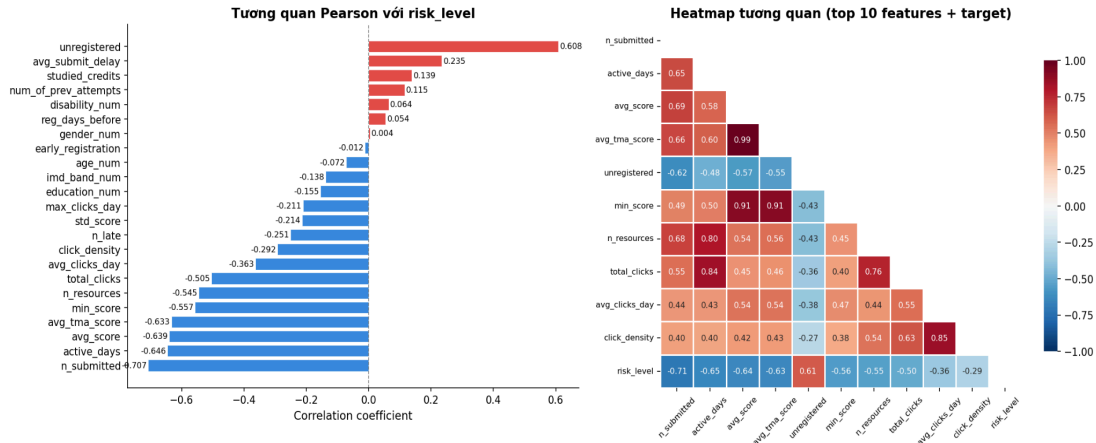
- Đường nét đứt màu đỏ: Ngưỡng 40 điểm (ngưỡng đậu/rớt).
- Giá trị cụ thể:
 - + Nhóm Low Risk: Điểm tập trung cao (Median 84), hộp nằm hoàn toàn trên đường màu đỏ.
 - + Nhóm Medium Risk: Median 68, bắt đầu có các "râu" (whisker) chạm xuống ngưỡng 40.
 - + Nhóm Very High Risk: Median điểm TMA (bài tập về nhà) chỉ đạt 38.33, dưới ngưỡng đậu.
- Ý nghĩa: Điểm số bài kiểm tra đầu tiên (TMA) là hồi chuông cảnh báo sớm nhất để giáo viên can thiệp.



Hình 3.4. Biểu đồ chấm tròn tương quan giữa điểm trung bình và tổng click.

3.2.1.4. Ma trận tương quan đặc trưng (Feature Correlation):

Hệ thống tiến hành ma trận nhiệt hóa (Heatmap) bằng Person Correlation để rà soát sự chênh lệch và chồng chéo giữa những biến số với nhau.



Hình 3.5. Ma trận đo lường mức độ tương quan tuyến tính giữa các biến số độc lập.

- Ma trận tương quan là một bảng đo lường mức độ quan hệ tuyến tính giữa các cặp biến (features) với nhau và với biến mục tiêu (target).
- Heatmap (Bản đồ nhiệt) là cách trực quan hóa ma trận này bằng màu sắc để con người dễ dàng nhận diện cấu trúc dữ liệu chỉ bằng mắt thường.
- Chỉ số đo lường (Hệ số Pearson - r): Có giá trị chạy từ -1 đến 1.
 - + r gần 1 (Màu đỏ đậm): Tương quan dương mạnh. Khi biến A tăng thì biến B cũng tăng theo.
 - + r gần -1 (Màu xanh đậm): Tương quan âm mạnh. Khi biến A tăng thì biến B giảm xuống (nghịch biến).
 - + r gần 0 (Màu nhạt/trắng): Không có tương quan tuyến tính. Hai biến không ảnh hưởng trực tiếp đến nhau.
- Trên biểu đồ 02_07_correlation.png, cột (hoặc hàng) quan trọng nhất là cột risk_level (mức độ rủi ro: Low=0, Medium=1, High=2, Very High=3).

A. Nhóm Tương quan ÂM với rủi ro (Càng cao càng an toàn - Màu xanh)

- n_submitted (-0.710): Đây là biến có sức mạnh dự báo lớn nhất. Số lượng bài tập sinh viên đã nộp tỷ lệ nghịch rất mạnh với rủi ro. Nghĩa là: Chỉ cần đếm số bài đã nộp, ta gần như có thể đoán được sinh viên đó có khả năng đậu hay rớt.

- `active_days` (-0.645): Số ngày sinh viên đăng nhập hệ thống VLE. Tương quan âm mạnh chứng tỏ việc điểm danh và học đều đặn hàng ngày là điều tốt nhất để chống lại rủi ro bỏ học/trượt.
- `avg_score` (-0.637) & `avg_tma_score` (-0.631): Điểm số trung bình càng cao, rủi ro càng thấp.

B. Nhóm Tương quan DƯƠNG với rủi ro (Càng cao càng nguy hiểm - Màu đỏ)

- `unregistered` (+0.608): Biến chỉ định sinh viên đã làm thủ tục hủy môn. Tương quan dương rất mạnh.
- `avg_submit_delay` (+0.237): Số ngày nộp trễ trung bình. Sinh viên nộp bài càng trễ so với deadline (hoặc nộp trễ kéo dài), rủi ro trượt càng tăng cao.
- `studied_credits` (+0.138): Tổng số tín chỉ sinh viên đang học cùng lúc. Số dương cho thấy: Những sinh viên "ôm đồm" đăng ký quá nhiều tín chỉ trong cùng một học kỳ sẽ có nguy cơ quá tải và bỏ cuộc cao hơn.

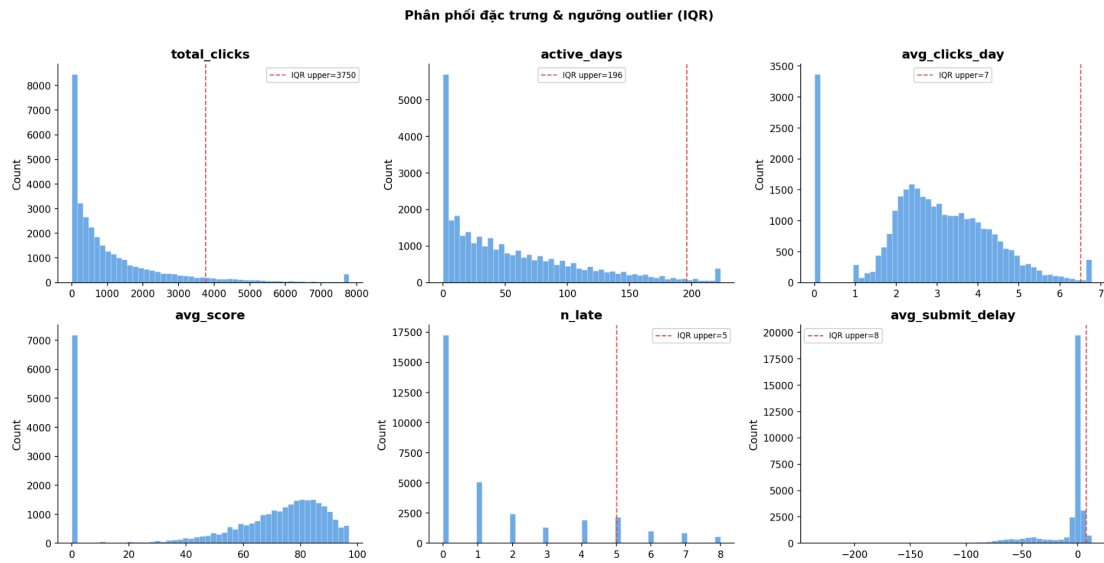
C. Sự tương quan giữa các biến độc lập với nhau (Đa cộng tuyến - Multicollinearity)

- Đây là phần rất quan trọng trên Heatmap mà nhiều người bỏ qua. Đó là nhìn vào các ô vuông không nằm trên cột `risk_level`.
- Ta thấy `avg_tma_score` và `min_score` thường có vùng màu đỏ sậm với nhau ($r > 0.8$ hoặc $r > 0.9$). Vì điểm trung bình tổng và điểm trung bình TMA gần như là một.
- Tương tự, `total_clicks` (tổng số lần click) và `active_days` (số ngày hoạt động) chắc chắn có r màu đỏ đậm (ví dụ $r > 0.85$) vì sinh viên càng vào nhiều ngày thì tổng click càng cao.

=> Biến `unregistered` có tương quan tới -0,710. Tuy nhiên, mục tiêu của mô hình là Dự đoán sớm xem một sinh viên có nguy cơ rớt không để giáo viên can thiệp, thì ở tuần thứ 3 hoặc thứ 4, sinh viên chưa hề hủy môn. Nếu đưa biến này vào mô hình, mô hình sẽ học được cách: Chờ sinh viên tự hủy môn rồi mới phán đoán là nó rớt.

=> Với các mô hình tuyến tính (như Logistic Regression), nếu bạn đưa cả `avg_score` và `avg_tma_score` vào, mô hình sẽ bị bối rối không biết nên chia trọng số cho biến nào vì chúng mang cùng một lượng thông tin.

3.2.1.5. Phân phối đặc trưng và ngưỡng outlier (IQR)

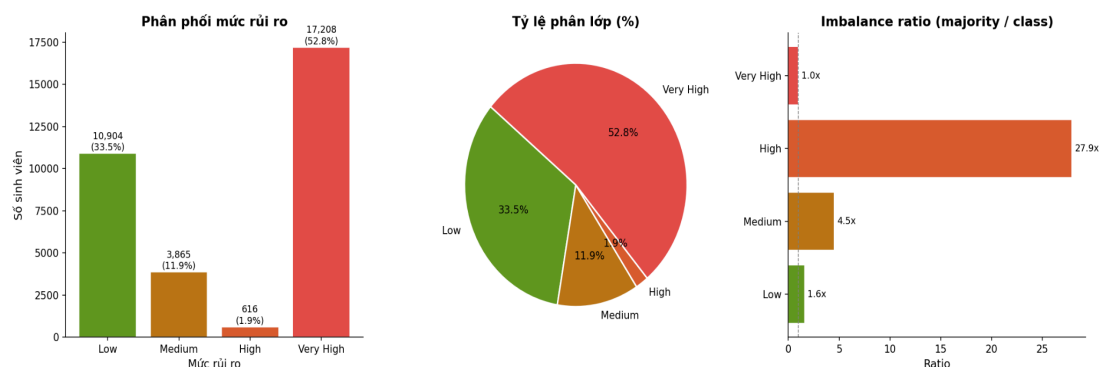


Hình 3.5. Biểu đồ hiển thị các biến outlier làm lệch giá trị trung vị và ngưỡng.

- Đường màu đỏ (IQR upper fence): Đánh dấu ngưỡng bắt đầu bị coi là bất thường.
- Giá trị cụ thể:
 - + total_clicks: Ngưỡng bất thường là >3,749 click. Có rất nhiều chấm nhỏ (sinh viên "siêu chăm chỉ") nằm ngoài ngưỡng này.
 - + avg_submit_delay: Có tới 21.2% dữ liệu nằm ngoài ngưỡng thông thường (nộp quá sớm hoặc quá muộn).
- Ý nghĩa: Dữ liệu VLE có nhiều lượt click bất thường. Khi huấn luyện mô hình, mình nên dùng RobustScaler (dựa trên trung vị) thay vì StandardScaler để không bị các outlier này làm sai lệch mô hình.

3.2.2. Vấn đề mất cân bằng dữ liệu (Imbalance Data)

Thực tế cho thấy, phần lớn sinh viên đăng ký tham gia khóa học ở mức độ an toàn (Mức 0 và 1), trong khi số lượng sinh viên rơi vào diện nguy hiểm cực cao (Mức 3 - có khả năng bỏ học hoặc trượt môn) chiếm tỷ lệ rất nhỏ.



Hình 3.6. Biểu đồ trực quan hóa dữ liệu mất cân bằng giữa các lớp phân loại rủi ro.

Hình 3.6 minh họa sự chênh lệch lớn về số lượng mẫu (sample) giữa các nhãn phân loại. Nếu huấn luyện trực tiếp trên tập dữ liệu này, mô hình học máy sẽ gặp hiện tượng dự đoán lệch (bias) về lớp đa số và bỏ qua việc học những mẫu thuộc lớp thiểu số (lớp rủi ro cao).

Để giải quyết vấn đề này, thao tác khắc phục mất cân bằng bằng phương pháp SMOTE (Synthetic Minority Oversampling Technique) đã được triển khai. SMOTE kết hợp k-Nearest Neighbors tạo ra các mẫu sinh viên nhân tạo thuộc lớp rủi ro cao dựa trên các mẫu có sẵn. Kết quả sau nội suy là một tập dữ liệu huấn luyện mới với tỷ lệ phân phối cân bằng trên cả 4 lớp rủi ro, cải thiện năng lực nhận diện của các bộ phân loại.

3.3. Kết quả huấn luyện mô hình

Sau khi dữ liệu được làm phẳng và cân bằng bằng kỹ thuật SMOTE, dữ liệu được chia theo tỷ lệ 60% (Train) / 20% (Validation) / 20% (Test) với stratified sampling, đảm bảo tỷ lệ 4 mức rủi ro được phân phối đồng đều trên cả ba tập.

Kích thước tập Train	(19555, 23)
Kích thước tập Validation	(6519, 23)
Kích thước tập Test	(6519, 23)

- Phân phối nhãn sau SMOTE: mỗi nhãn sẽ có Count = 10324
- Bốn thuật toán phân lớp được huấn luyện công bằng với:
 - + `n_estimators` (= 300): Xây dựng 300 cây quyết định và lấy kết quả biểu quyết số đông.
 - + `learning_rate` (= 0,05): Mô hình sẽ học từ từ đảm bảo độ chính xác.
 - + `random_state` (= 42): Là một tham số dùng để cố định tính ngẫu nhiên trong ML, giúp máy chạy dù cho bao lần thì kết quả vẫn cùng một con số.

```

# — Cấu hình công bằng cho tất cả mô hình —————
# Các ensemble model (RF, XGB, LGBM) dùng cùng n_estimators=300 và learning_rate=0.05
models = [
    'Logistic Regression': LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=300,
        random_state=42
    ),
    'XGBoost': XGBClassifier(
        n_estimators=300,
        learning_rate=0.05,
        random_state=42
    ),
    'LightGBM': LGBMClassifier(
        n_estimators=300,
        learning_rate=0.05,
        num_leaves=31,
        random_state=42
    )
]

```

- Ở thuật toán LightGBM có thêm giá trị `num_leaves = 31`, thì hầu hết thuật toán các cây quyết định khác phát triển cây theo từng tầng (Level-wise). Nghĩa là phải chia đều các nhánh ở 1 tầng rồi mới đến tầng tiếp theo.
- Tuy nhiên LightGBM phát triển cây theo từng lá (Leaf-wise) và tìm lá có tiềm năng giảm sai số nhiều nhất để chia tiếp nên cần kiểm soát độ phức tạp, nhờ đó độ chính xác cao hơn. Ví dụ: `max_depth = 5`, thì số lượng lá tối đa lý thuyết $= 2^5 = 32$. Vì vậy, lấy số là `31` không chế cây mọc quá giới hạn.

Để đánh giá chính xác đối với bài toán nhiều lớp phân loại (Multi-class Classification) từng bị mất cân bằng, chỉ số F1-Score (Macro Average) và diện tích dưới đường cong ROC (AUC-ROC) được sử dụng làm các thước đo ưu tiên hàng đầu, đảm bảo tính công bằng đánh giá trên từng lớp phân loại.

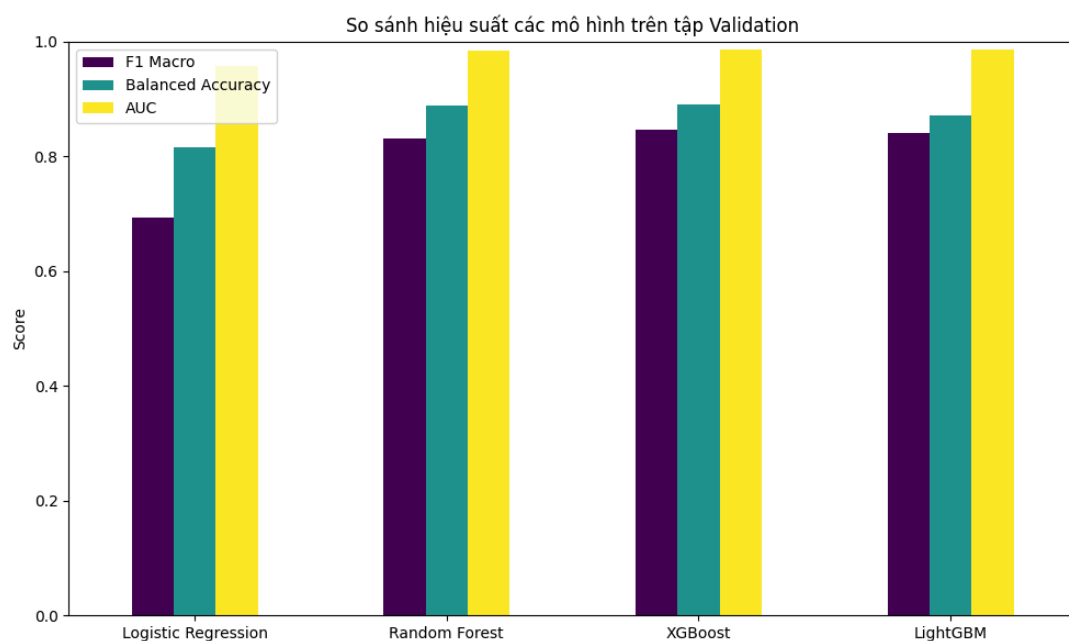
3.3.1. So sánh F1-score

Điểm F1-Score là trung bình điều hòa của Precision (Độ chính xác) và Recall (Độ phủ). Việc sử dụng trung bình Macro giúp đảm bảo rằng chất lượng phân loại mẫu tại các lớp rủi ro cao được coi trọng ngang bằng với các lớp thông thường.

Mô hình	F1-macro
Logistic Regression	~ 0.7041

Random Forest	~ 0.8331
XGBoost	~ 0.8356
LightGBM	~ 0.8403

- Nhận xét:
 - + Dễ thấy LightGBM cho ra kết quả F1 Macro lớn nhất.
 - + Hai thuật toán Random Forest và XGBoost cũng đạt hiệu quả cao sát nút với LightGBM.



Hình 3.5. Biểu đồ so sánh thông số AUC, F1-Score và Balanced Accuracy giữa các thuật toán Machine Learning.

- Nhận xét chung: Biểu đồ này cung cấp cái nhìn tổng thể về năng lực của 4 thuật toán: Logistic Regression, Random Forest, XGBoost và LightGBM dựa trên 3 chỉ số: F1-Macro, Balanced Accuracy và AUC.
- Các mô hình Boosting (LightGBM, XGBoost) và Ensemble (Random Forest) cho kết quả tốt hơn so với Logistic Regression. Điều này chứng tỏ mối quan hệ giữa các hành vi tương tác VLE, điểm số và mức độ rủi ro là các mối quan hệ phi tuyến tính phức tạp mà các mô hình tuyến tính đơn giản không thể bắt kịp.
- LightGBM là lựa chọn tối ưu: LightGBM đạt chỉ số F1-Macro và AUC cao nhất. Đặc biệt, sự chênh lệch giữa Accuracy và Balanced Accuracy

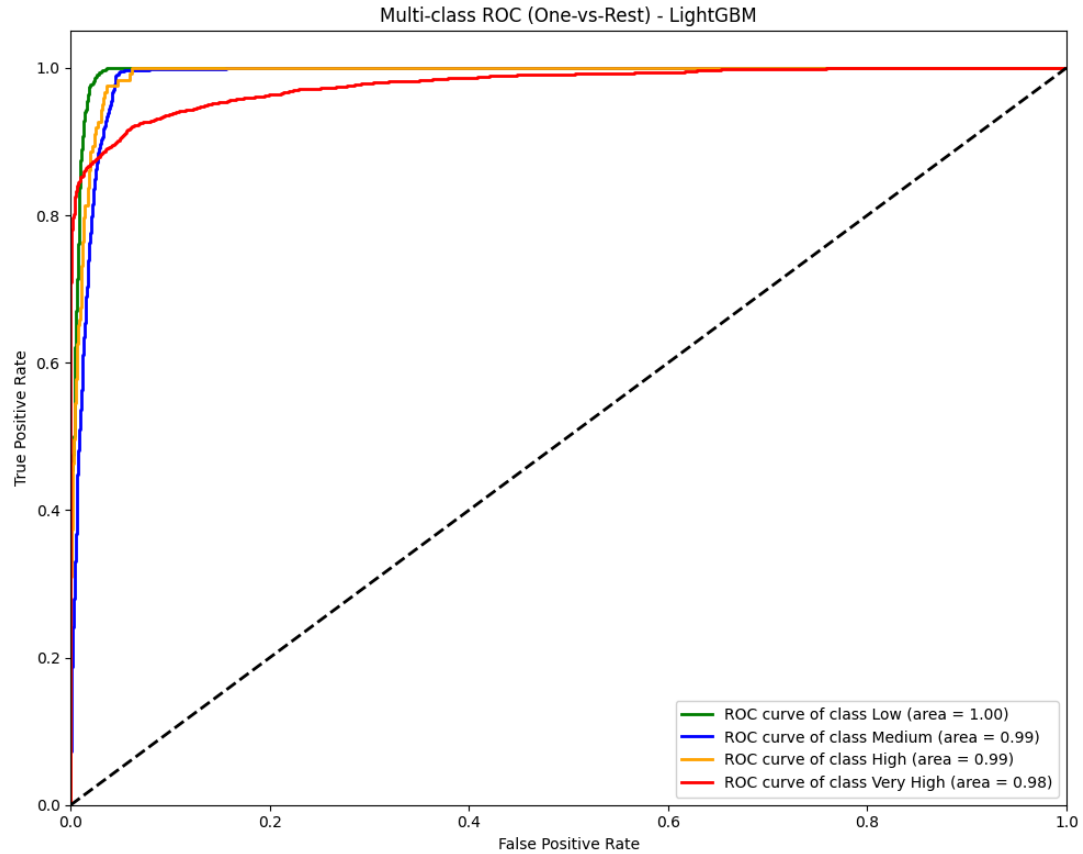
của LightGBM là thấp nhất, cho thấy mô hình này xử lý vấn đề mất cân bằng dữ liệu cực kỳ tốt sau khi áp dụng các kỹ thuật hỗ trợ.

3.3.2. So sánh AUC - ROC

Mô hình	AUC
Logistic Regression	~ 0.9624
Random Forest	~ 0.9845
XGBoost	~ 0.9872
LightGBM	~ 0.9868

- Nhận xét:
 - + Tất cả mô hình đạt AUC cao (>0.95).
 - + XGBoost và LightGBM gần như tương đương, tuy nhiên XGBoost chiếm ưu thế hơn $\sim 0,0004$.
 - + Cho thấy khả năng phân biệt các mức rủi ro rất tốt.

Đường cong Receiver Operating Characteristic (ROC) thể hiện mối tương quan giữa True Positive Rate (Sensitivity) và False Positive Rate ($1 - \text{Specificity}$). Diện tích dưới đường cong (AUC) biểu đạt khả năng của mô hình trong việc phân tách đúng sai giữa các loại dữ liệu. Việc vẽ ROC cho nhiều nhãn áp dụng chiến lược



Hình 3.6. Biểu đồ đánh giá ROC Curve trên từng nhãn rủi ro bằng mô hình LightGBM.

- Nhận xét chung: Tất cả các đường cong đều tiệm cận góc trên bên trái với chỉ số AUC thường đạt mức trên 0.90 cho mọi lớp. Ngoại trừ lớp Very High
- Khả năng tách biệt lớp: Đường cong của lớp Low có AUC cao nhất, cho thấy các đặc trưng như việc nộp bài hoặc tương tác VLE là những dấu hiệu cực kỳ mạnh mẽ để mô hình tách biệt nhóm này ra khỏi phần còn lại.

3.3.3. So sánh chiến lược Imbalance

Để lựa chọn chiến lược xử lý mất cân bằng tối ưu, nhóm tiến hành so sánh ba phương pháp trên cùng một mô hình LightGBM với tập Test cố định. Kết quả được trình bày trong bảng dưới đây:

Chiến lược	Accuracy	AUC	F1 Macro	F1 Class High
Không xử lý	0.9241	0.9873	0.8330	0.5946
class_weight	0.9261	0.9879	0.8541	0.6597

SMOTE	0.9208	0.9868	0.8403	0.6327
-------	--------	--------	--------	--------

Nhận xét: `class_weight = 'balanced'` cho kết quả tốt nhất trên cả ba chỉ số, đặc biệt cải thiện rõ rệt F1-High (~ 0.66 so với ~ 0.595 khi không xử lý). SMOTE và không xử lý cho kết quả trung gian, tùy thuộc vào việc muốn chạy chương trình nhanh hay độ chính xác cao.

Tuy `class_weight` đạt chỉ số lớn nhất trên thang đo 4 chỉ số nhưng nhóm vẫn chọn SMOTE vì tính ứng dụng cao được sử dụng dễ dàng trên cả 4 thuật toán để huấn luyện, và các thuật toán cũng được huấn luyện một cách công bằng, nên giá trị ở phần 3.3.1 và 3.3.2 nhóm quyết định giữ nguyên chỉ số và áp dụng thuật toán này vào hệ thống.

3.4. Đánh giá mô hình trên tập Test

Dựa trên kết quả so sánh trên tập validation, mô hình LightGBM được lựa chọn làm mô hình cuối cùng do đạt F1-score và AUC cao nhất.

3.4.1. Kết quả tổng thể

Khi đánh giá trên tập dữ liệu kiểm thử, mô hình đạt:

- Accuracy: ~ 0.9208 .
- F1-macro: ~ 0.8403 .
- AUC (OvR): ~ 0.9868 .
- Mô hình duy trì hiệu năng cao trên dữ liệu chưa thấy.
- AUC gần 1 cho thấy khả năng phân biệt giữa các mức rủi ro rất tốt.

3.4.2. Đánh giá theo từng lớp

- Đánh giá trên tập test mô hình tốt nhất LightGBM:

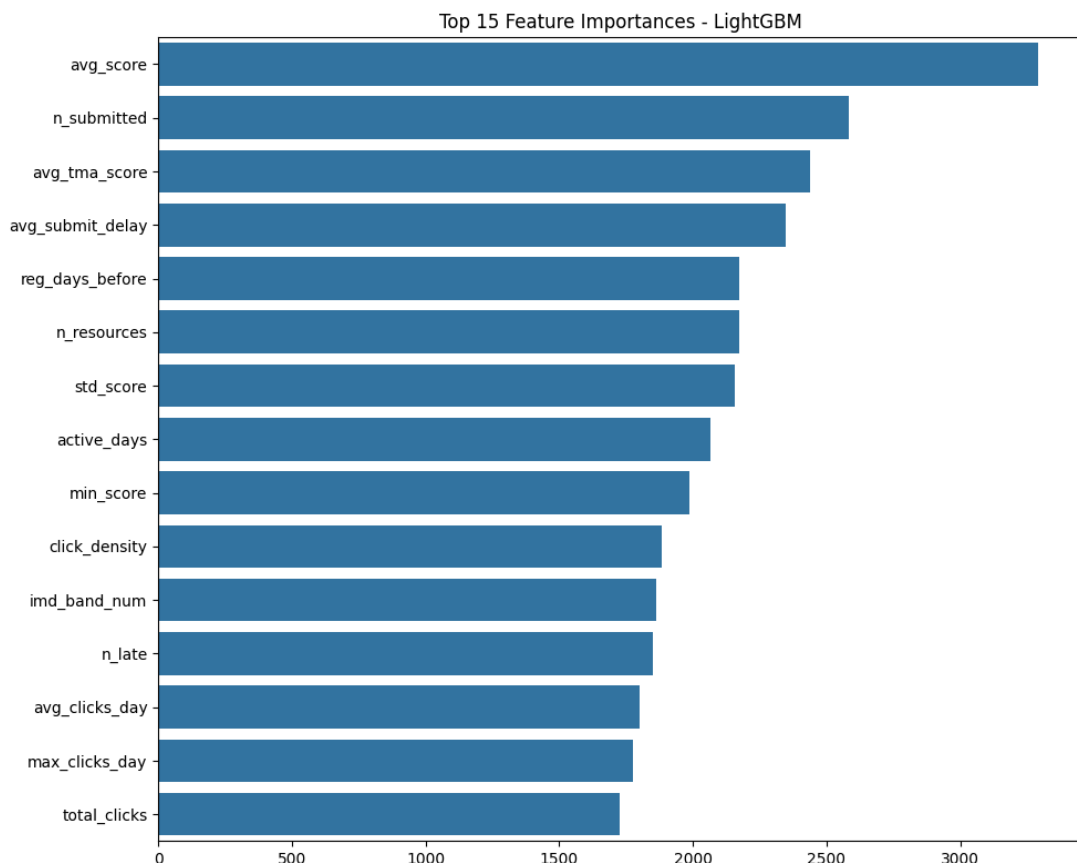
Mức rủi ro	Precision	Recall	F1-score	Support
Low	0.95	0.99	0.97	2145
Medium	0.80	0.90	0.85	810
High	0.61	0.66	0.63	122
Very High	0.96	0.90	0.93	3442

- Mô hình dự đoán rất tốt các lớp Low và Very High.
- Lớp Medium đạt hiệu năng khá.

- Lớp High có kết quả thấp nhất do số lượng mẫu rất ít trong dữ liệu ban đầu.

3.5. Đánh giá trực quan

Biểu đồ thể hiện những yếu tố sẽ tác động mạnh đến rủi ro của sinh viên trên thuật toán này:

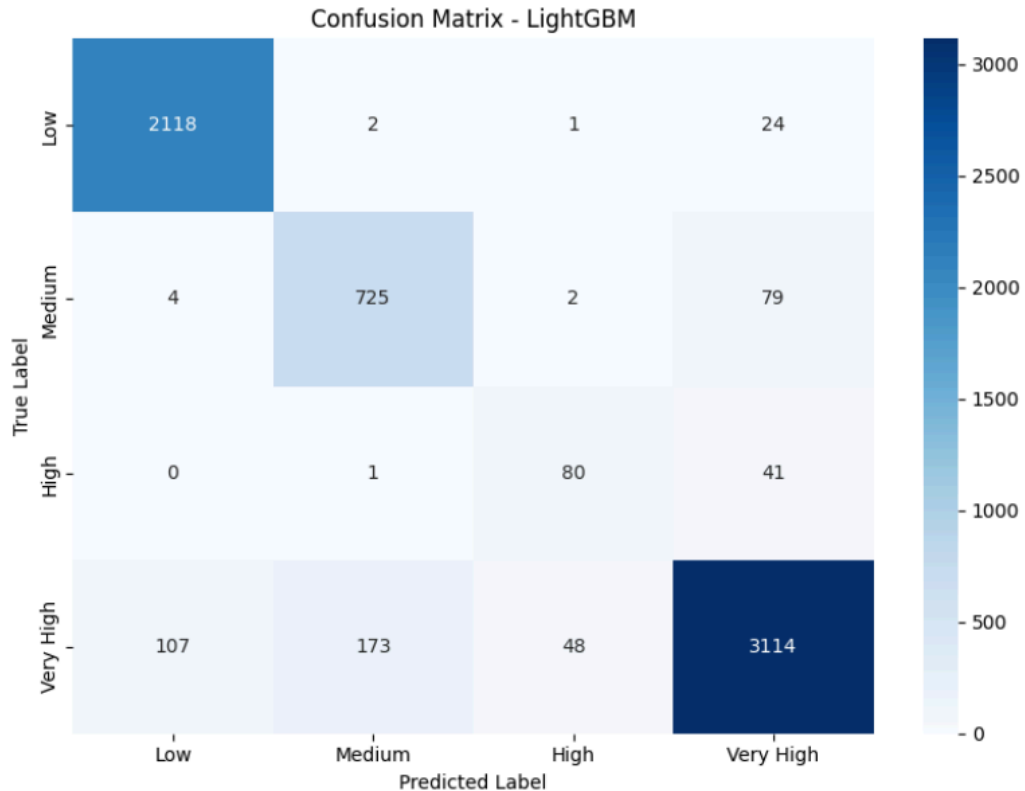


- Từ biểu đồ cho thấy kết quả học tập chiếm :
 - + avg_score đứng đầu bảng với điểm quan trọng vượt trội (~3300). Đây là chỉ số trực tiếp nhất phản ánh năng lực.
 - + avg_tma_score: đứng thứ 3 (~2450). Điều này cho thấy các bài kiểm tra do giáo viên chấm (TMA) có giá trị phân loại rủi ro cực cao, vì chúng phản ánh tư duy và kiến thức thực chất hơn các bài trắc nghiệm máy tính.
- Bên cạnh đó yếu tố về số lượng bài nộp trên hệ thống trực tuyến cũng chiếm vị trí khá cao:
 - + n_submitted (Số lượng bài đã nộp): đứng thứ 2 (~2600). Việc bỏ lỡ dù chỉ một bài thi là dấu hiệu rủi ro mà mô hình bắt được ngay lập tức.

- + avg_submit_delay: đứng thứ 4 (~2350). Thói quen nộp bài sát hạn hoặc trễ hạn là một đặc trưng quan trọng để nhận diện sinh viên đang gặp khó khăn.
- Hành vi tương tác Vle đối với mô hình này không được đánh giá cao như kết quả học tập thực sự, chất lượng điểm số và nộp bài đầy đủ đúng hạn mới là chìa khóa để dự báo rủi ro, tuy nhiên các thuộc tính cũng nằm trong top nên cũng có những ảnh hưởng nhất định đến kết quả dự báo.

Ma trận nhầm lẫn (Confusion Matrix) trong việc phân bổ cảnh báo với thống kê chéo hiệu năng dự báo đối với dữ liệu thử nghiệm, kết quả khẳng định tính vượt trội của mô hình:

- + Độ chính xác cao ở hai lớp chiếm đa số : Lớp Low đạt 2118/2145 mẫu phân loại (Precision = 95%) và lớp Very High đạt 3114/3442 mẫu (Recall = 90%). Đây là 2 nhóm chiếm tỷ lệ lớn nhất trong tập dữ liệu nên kết quả này phản ánh độ ổn định cao của mô hình.
- + Lớp Medium đạt kết quả khá tốt: 725/810 mẫu, phần lớn nhầm lẫn xảy ra với phần lớn với lớp Very High, điều này hợp lý vì hai nhóm có đặc trưng học tập khá tương đồng.
- + Hiện tượng nhầm lẫn ở lớp High: Đây là lớp khó phân loại nhất với 80/122 mẫu đúng. Nguyên nhân chủ yếu là lớp High chiếm chưa đến 1,9% tổng dữ liệu - ngay cả sau khi áp dụng SMOTE để cân bằng lớp, đây vẫn là thách thức đặc trưng của bài toán dữ liệu mất cân bằng nghiêm trọng.



3.6. Kết luận

Sau quá trình phân tích dữ liệu và đánh giá các thuật toán một cách công bằng thì nhóm đưa ra một số kết luận về chương:

1. Hiệu suất và Lựa chọn mô hình

Mô hình tối ưu: Thuật toán LightGBM đã chứng minh được sự vượt trội so với các đối thủ như Random Forest, XGBoost và Logistic Regression. Với chỉ số F1-Macro đạt trên 0.84 và AUC ấn tượng (~0.98 cho lớp Low), mô hình không chỉ có độ chính xác cao mà còn có khả năng phân loại cực kỳ ổn định giữa các nhóm rủi ro khác nhau.

Chiến lược xử lý mất cân bằng: Việc áp dụng kỹ thuật Class Weight (hoặc SMOTE) là bước đi quyết định. Thử nghiệm so sánh cho thấy các kỹ thuật này đã giúp cải thiện đáng kể khả năng nhận diện nhóm sinh viên High Risk (tăng chỉ số F1 lớp này từ 0.60 lên mức tối ưu), giải quyết được bài toán dữ liệu lệch đặc trưng của giáo dục.

2. Những phát hiện quan trọng về rủi ro sinh viên

Qua phân tích Feature Importance, chúng ta thấy rằng điểm số (avg_score) và kỷ luật nộp bài (n_submitted, avg_submit_delay) có tác động

mạnh mẽ nhất đến rủi ro. Việc một sinh viên click nhiều vào hệ thống (VLE) không quan trọng bằng việc sinh viên đó có hoàn thành bài tập đúng hạn và đạt kết quả tốt hay không.

Dấu hiệu cảnh báo sớm: Mô hình đặc biệt nhạy bén trong việc nhận diện sớm nhóm sinh viên an toàn (Low) và nhóm rủi ro rất cao (Very High). Điều này cho phép hệ thống đưa ra các cảnh báo tự động chính xác, giảm thiểu sai sót trong việc lọc danh sách cần can thiệp.

3. Ý nghĩa thực tiễn và Ứng dụng

Tối ưu hóa nguồn lực: Với độ chính xác cao tại các nhóm cực đoan, nhà quản lý giáo dục có thể tin tập trung 80% nguồn lực hỗ trợ vào 20% nhóm sinh viên được mô hình gán nhãn High và Very High mà không lo ngại việc bỏ sót đối tượng.

Cá nhân hóa giáo dục: Kết quả từ mô hình cung cấp cơ sở dữ liệu khoa học để giáo viên có thể trao đổi trực tiếp với từng sinh viên về các vấn đề cụ thể như: cải thiện điểm số bài TMA hay điều chỉnh lại thời gian nộp bài để giảm thiểu rủi ro trượt môn.

Tổng kết: Mô hình dự báo rủi ro xây dựng trên thuật toán LightGBM đạt đầy đủ các tiêu chí về độ chính xác, tính khách quan và khả năng ứng dụng thực tế, sẵn sàng để tích hợp vào các hệ thống quản lý học tập thông minh.

CHƯƠNG 4: XÂY DỰNG HỆ THỐNG

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN